
CHAPTER 6

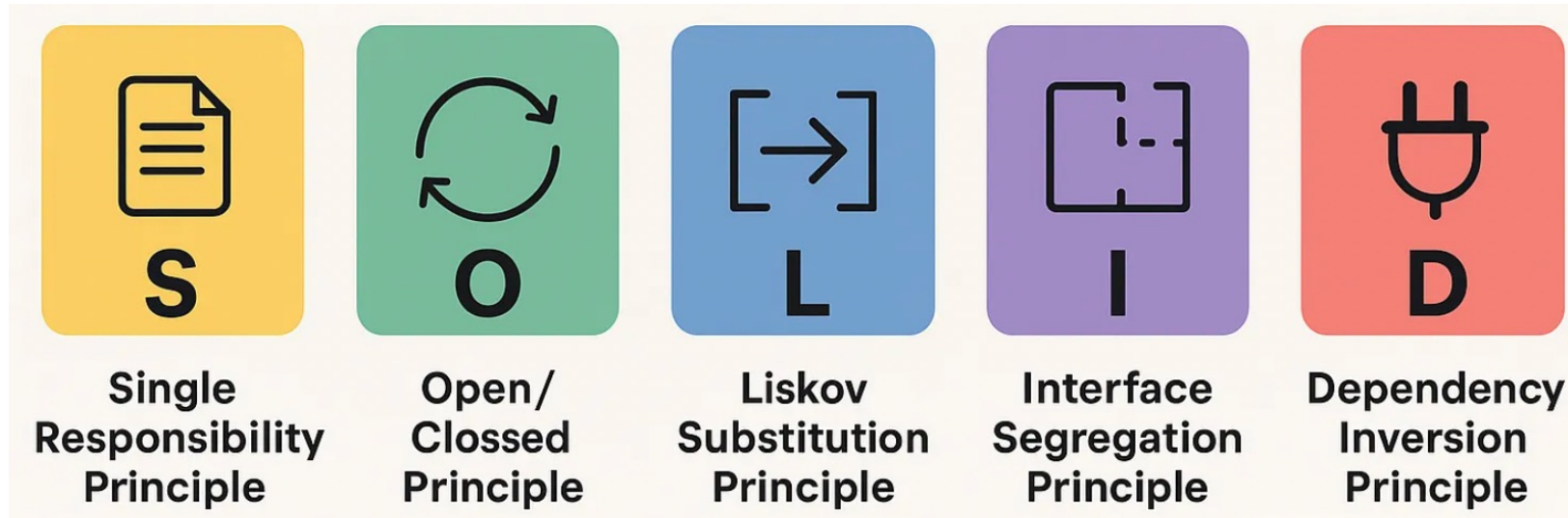
DESIGN PATTERN

Chapter 6: DESIGN PATTERN

OUTLINE

1. Understanding SOLID Principles
2. What is a design?
3. Why do we need design?
4. What is a pattern?
5. What is a design pattern?
6. Types of design patterns

Understanding SOLID Principles



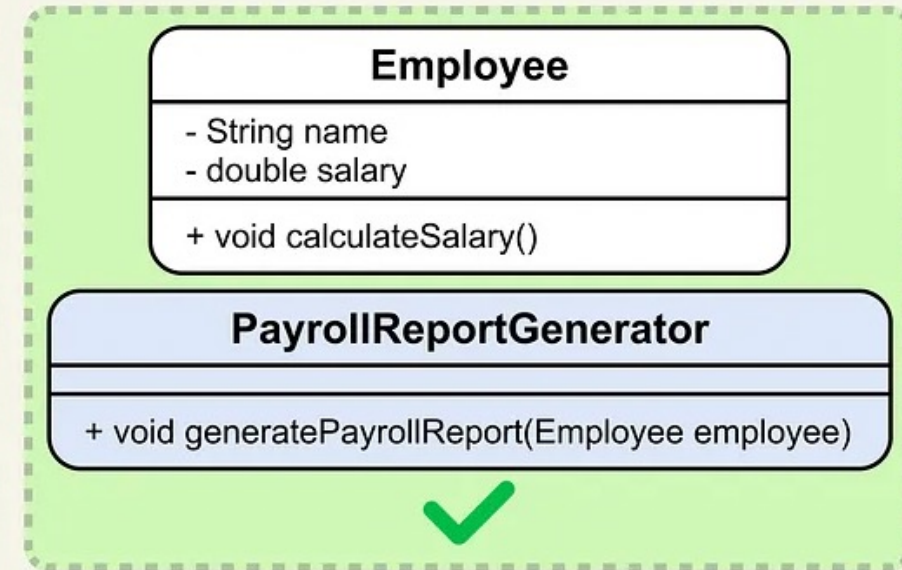
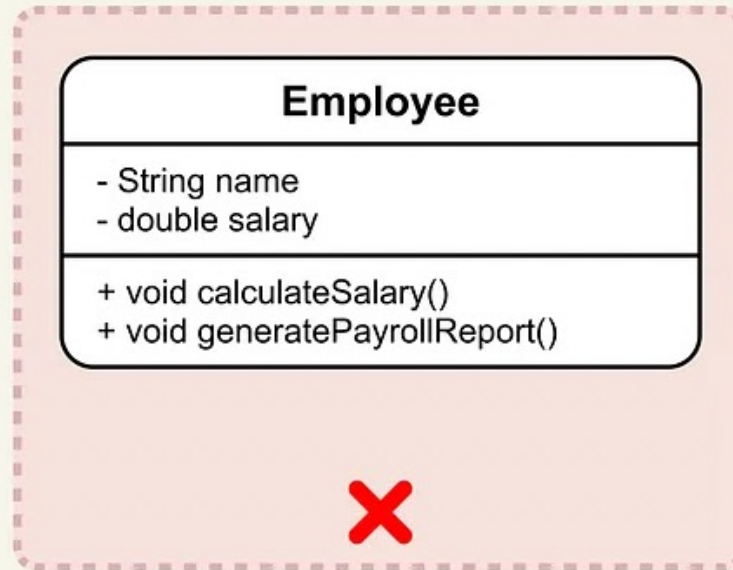
The SOLID principles are not rules (quy tắc). They are not laws (luật lệ). They are not perfect truths. They are statements on the order of “An apple a day keeps the doctor away.” This is a good principle, it is good advice, but it’s not a pure truth, nor is it a rule.

Robert Cecil Martin - Uncle Bob

Understanding SOLID Principles (cont.)

Single Responsibility Principle (SRP)

A class should have only one reason to change, meaning it should have a single, well-defined responsibility.

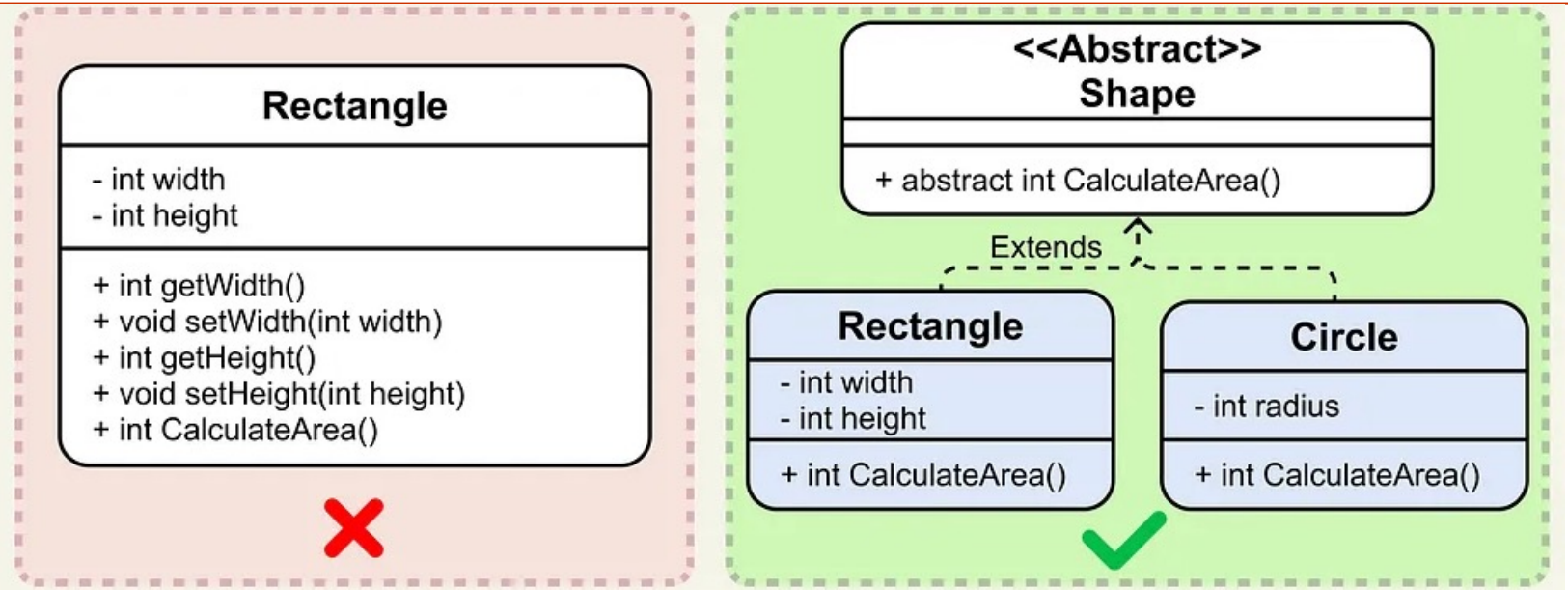


- This makes your code easier to maintain and test.
- Ex: if a class handles both calculate salary and reporting, changes in reporting requirements could affect calculate salary logic, leading to bugs.

Understanding SOLID Principles (cont.)

Open/Closed Principle (OCP)

Software entities (e.g., classes, modules) should be open for extension but closed for modification.

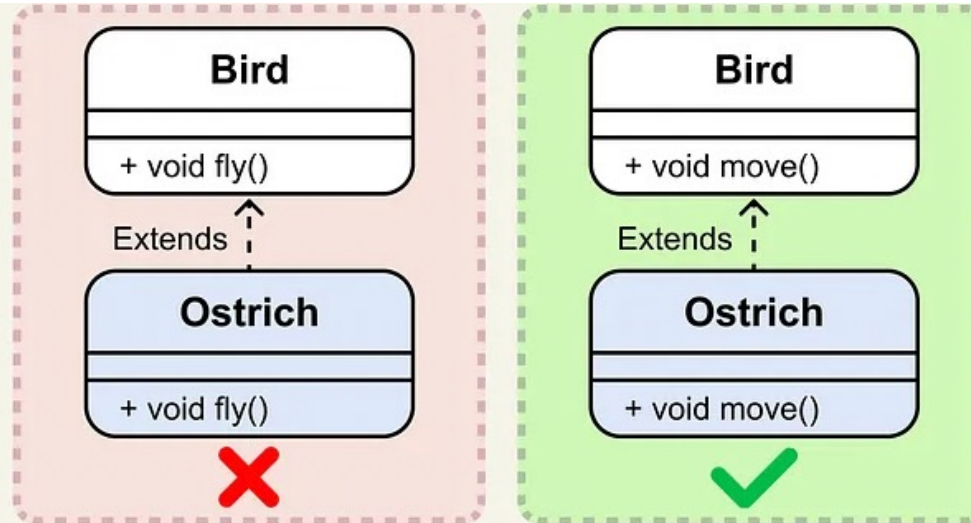


→ The Open/Closed Principle (OCP) states that software entities, such as classes, should be open for extension but closed for modification. This means you can add new functionality without altering existing code.

Understanding SOLID Principles (cont.)

Liskov Substitution Principle (LSP)

Subtypes (derived classes) must be substitutable for their base types (parent classes) without altering the correctness of the program.

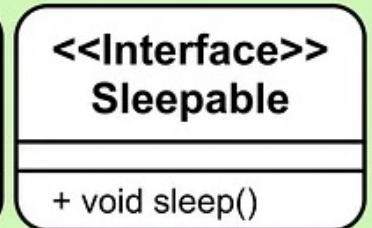
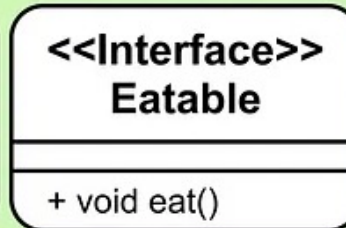
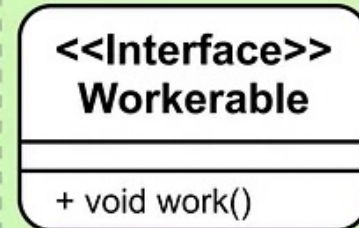
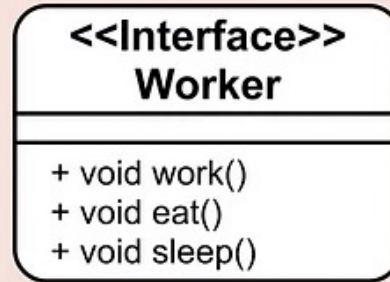


→ The Liskov Substitution Principle (LSP) states that objects of a derived class should be able to replace objects of the base class without affecting the correctness of the program.

Understanding SOLID Principles (cont.)

Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they don't use.

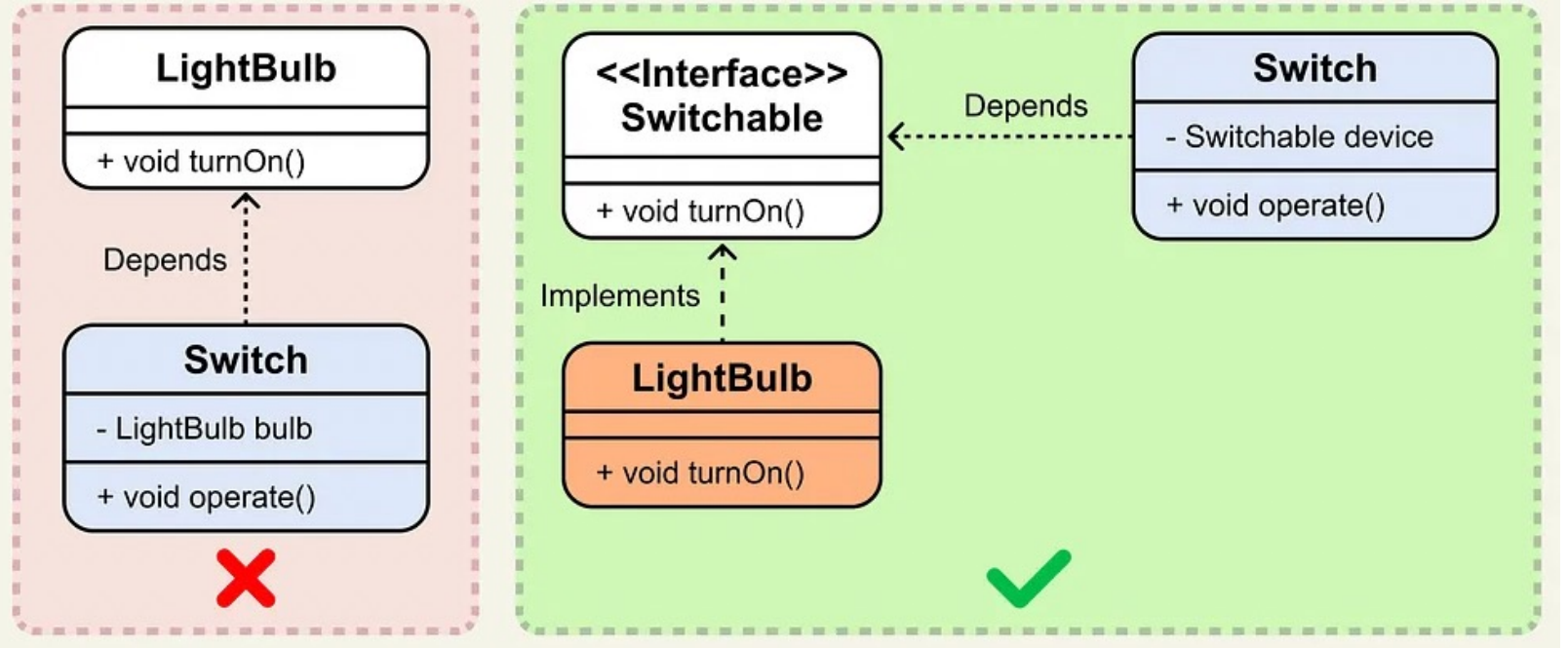


→ Clients should not be forced to depend on interfaces they do not use. Split large interfaces into smaller, more specific ones so classes only implement what they need. This keeps your code flexible and easier to maintain

Understanding SOLID Principles (cont.)

Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions.



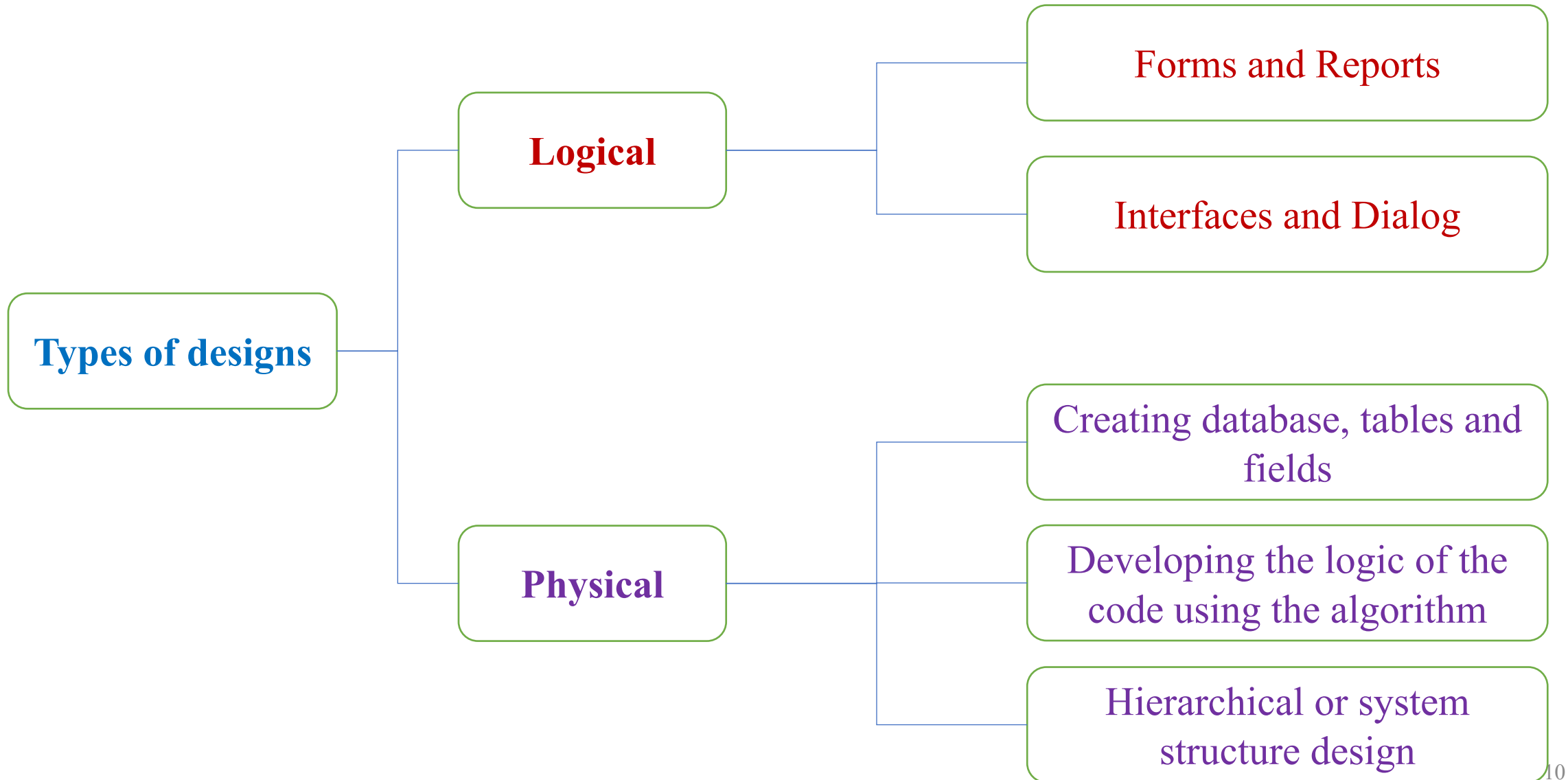
→ High-level modules should not depend on low-level modules. Both should depend on abstractions. This makes your code more flexible and easier to test, as you can swap implementations without changing the high-level logic.

What is a design?

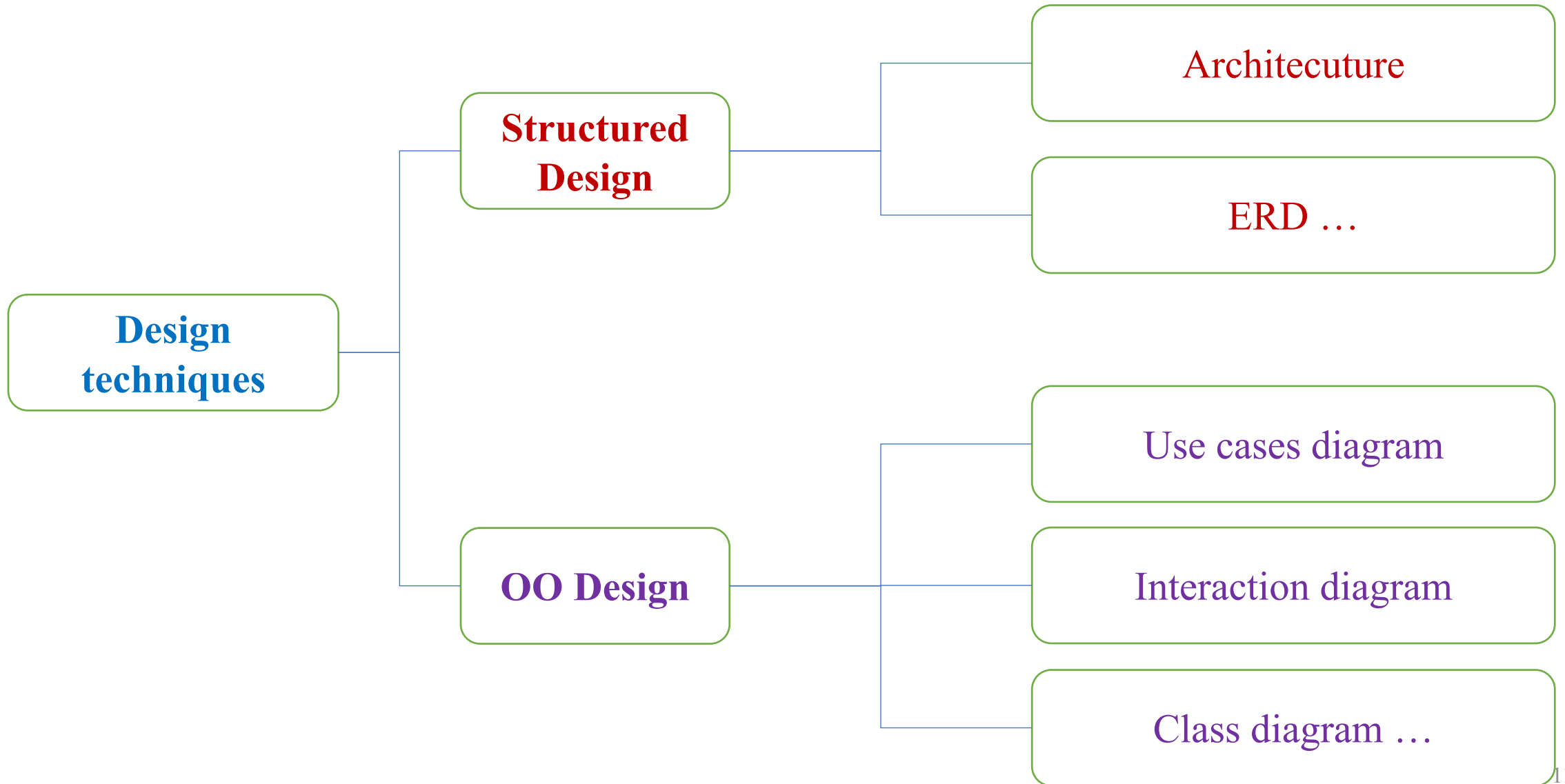
- Formulate the maximum alternative solutions of the problem and select the best one.
- Design possibilities:
 - Algorithmic strategy
 - Decomposition of a system
 - Abstract function hierarchy



Type of designs



Design techniques



Why do we need design?

- Provides an overall picture of the system
- Allows us to focus on one thing at a time
- Provides concept reuse (NOT code reuse)
- Simplify implementation
- Assesses quality

What is a design pattern?

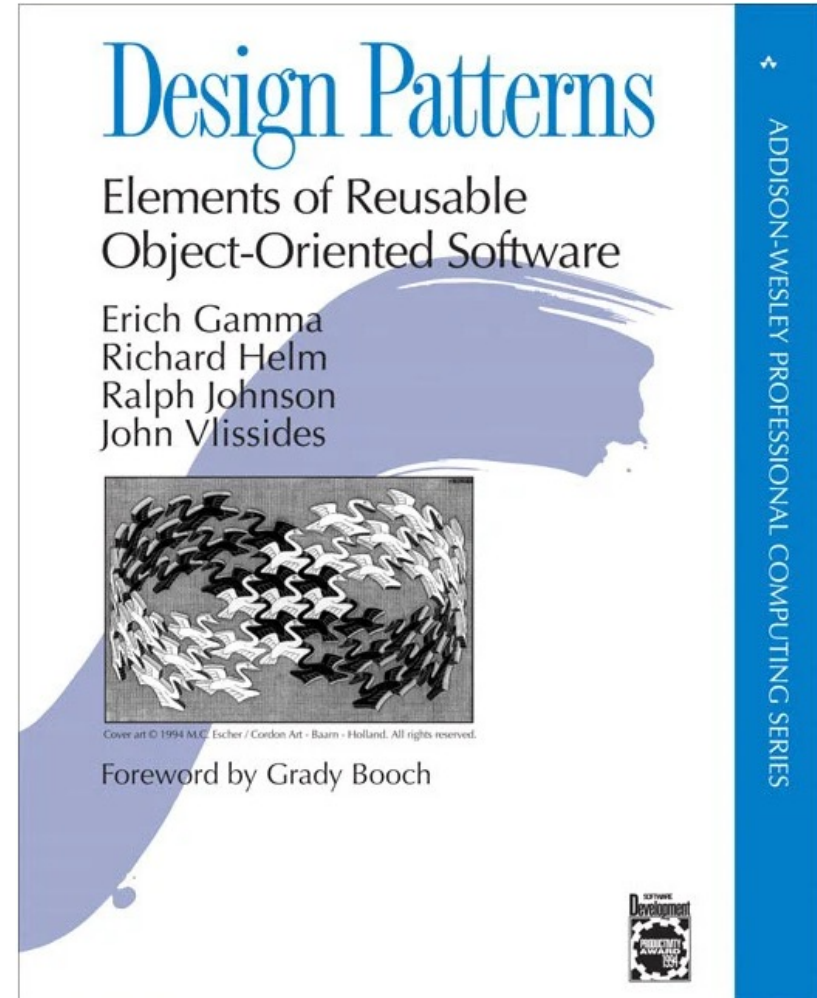
- In software engineering, a design pattern is a general repeatable solution to a commonly occurring problem in Software design
- It isn't a finished design that can be transformed directly into code, but a description or template for how to solve a problem that can be used in many different situations

Design pattern - Usage

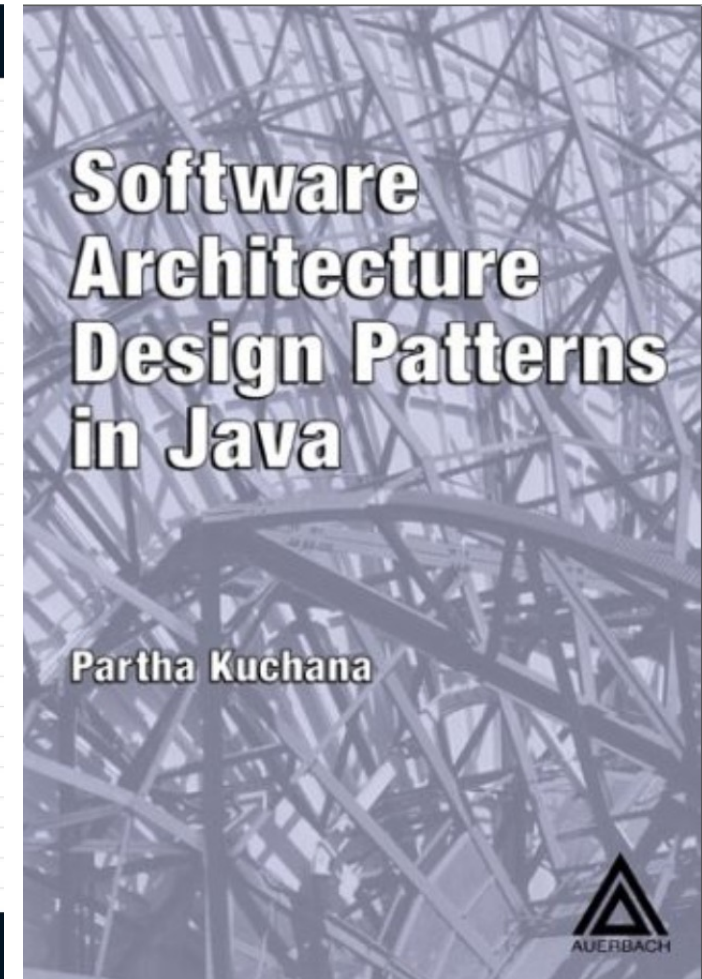
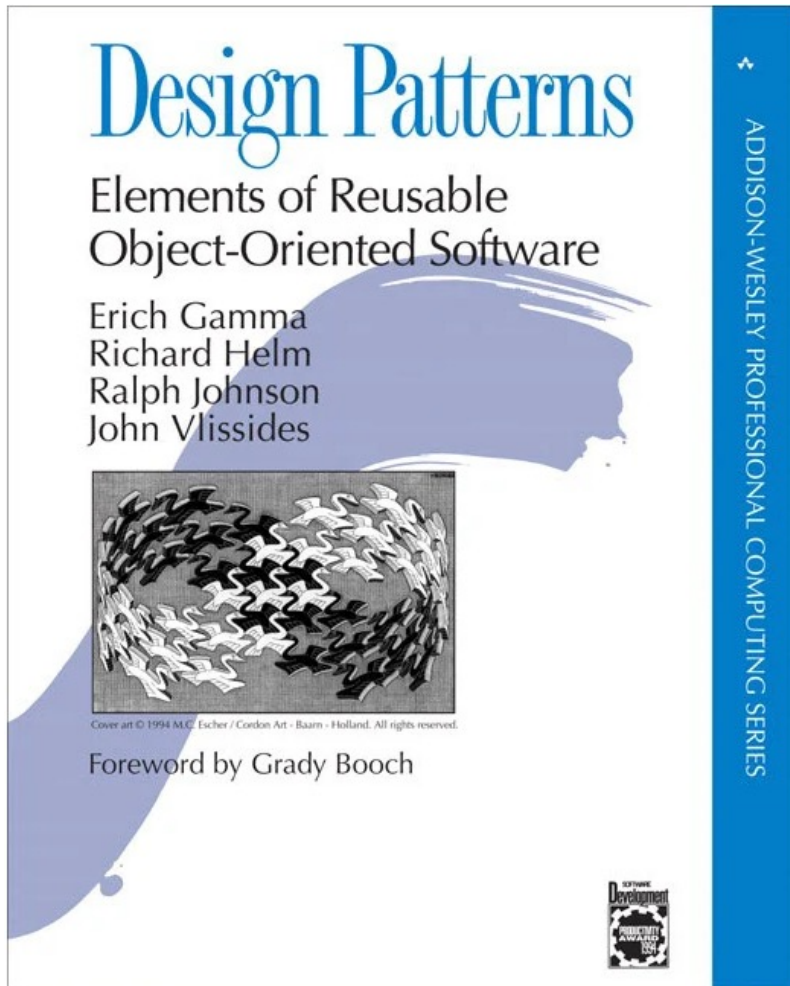
- Provide general solutions, documented in a format that doesn't require specifics tied to a particular problem
- Can speed up the development process by providing tested, proven development paradigms
- Help you benefit from the experience of fellow developers
- Prevent subtle issues that can cause major problems
- Improve code readability for codes and architects familiar with them

Gang of Four – GoF Book

- Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley Publishing Company, 1994
- Written by this “gang of four”
 - Dr. Erich Gamma
 - Dr. Richard Helm
 - Dr. Ralph Johnson
 - Dr. John Vlissides



Books



GoF Design Patterns

- The Gang of Four are the four authors of the book << Design Patterns: Elements Of Reusable Object-Oriented Software>>
- Defined 23 design patterns for recurrent design issues, called GoF design pattern
- Classified by purpose:
 - *Creational*: concerns the **creation process** of objects and classes
 - *Structural*: concerns the **composition** of classes and objects
 - *Behavioral*: characterizes the **interaction and responsibility** of objects and classes
- Classified by scope:
 - *Class*: relationship between classes and subclasses, defined statically
 - *Object*: object relationship, dynamic

GoF Design Patterns

Scope	Creational Patterns	Structural Patterns	Behavioral Patterns
Class	▪ Factory method	▪ Adapter	▪ Interpreter ▪ Template method
Object	▪ Abstract factory ▪ Builders ▪ Prototype ▪ Singleton	▪ Adapter ▪ Bridge ▪ Composite ▪ Decorator ▪ Facade ▪ Proxy	▪ Chain of responsibility ▪ Command ▪ Iterator ▪ Mediator ▪ Memento ▪ Observer ▪ State ▪ Strategy ▪ Visitor

GoF Design Patterns (cont.)

1. ***Singleton***: Ensure a class has only one instance, and provide a global point of access to it.
2. ***Factory***: Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
3. ***Abstract Factory***: Provide an interface for creating families of related or dependent objects without specifying their concrete classes.
4. ***Builder***: Separate the construction of a complex object from its representation allowing the same construction process to create various representations.
5. ***Prototype***: Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.

GoF Design Patterns (cont.)

6. ***Adapter***: Provides an interface between two unrelated entities so that they can work together. It acts as a bridge between two incompatible interfaces.
7. ***Bridge***: Decouple an abstraction from its implementation allowing the two to vary independently.
8. ***Composite***: Takes a group of objects into a single object. This pattern is perfect for representing part-whole hierarchies, such as a graphics system where shapes like circles and squares can be grouped into a larger diagram.
9. ***Proxy***: Provides a placeholder interface to an underlying object to control access, reduce cost, or reduce complexity. This pattern is commonly used for lazy initialization, access control, or logging, effectively managing resources by acting as an intermediary.
10. ***Facade***: Helps in Simplifying the complex system interfaces.

GoF Design Patterns (cont.)

- 11. *Flyweight*:** Caching and reusing object instances, used with immutable objects. Reduces the cost of complex object models. This pattern is effective in scenarios where you need to manage a large number of similar objects, such as in a string pool, by sharing common data to minimize memory usage.
- 12. *Observer or Publish/subscribe*:** Define a one-to-many dependency between objects where a state change in one object results with all its dependents being notified and updated automatically.
- 13. *State*:** Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.
- 14. *Strategy*:** Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

GoF Design Patterns (cont.)

- 15. *Command*:** Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.
- 16. *Chain of Responsibility*:** is a behavioral design pattern that allows an object to pass a request along a chain of handlers.
- 17. *Iterator*:** Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
- 18. *Mediator*:** Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.
- 19. *Memento*:** Without violating encapsulation, capture and externalize an object's internal state allowing the object to be restored to this state later.
- 20. ...**

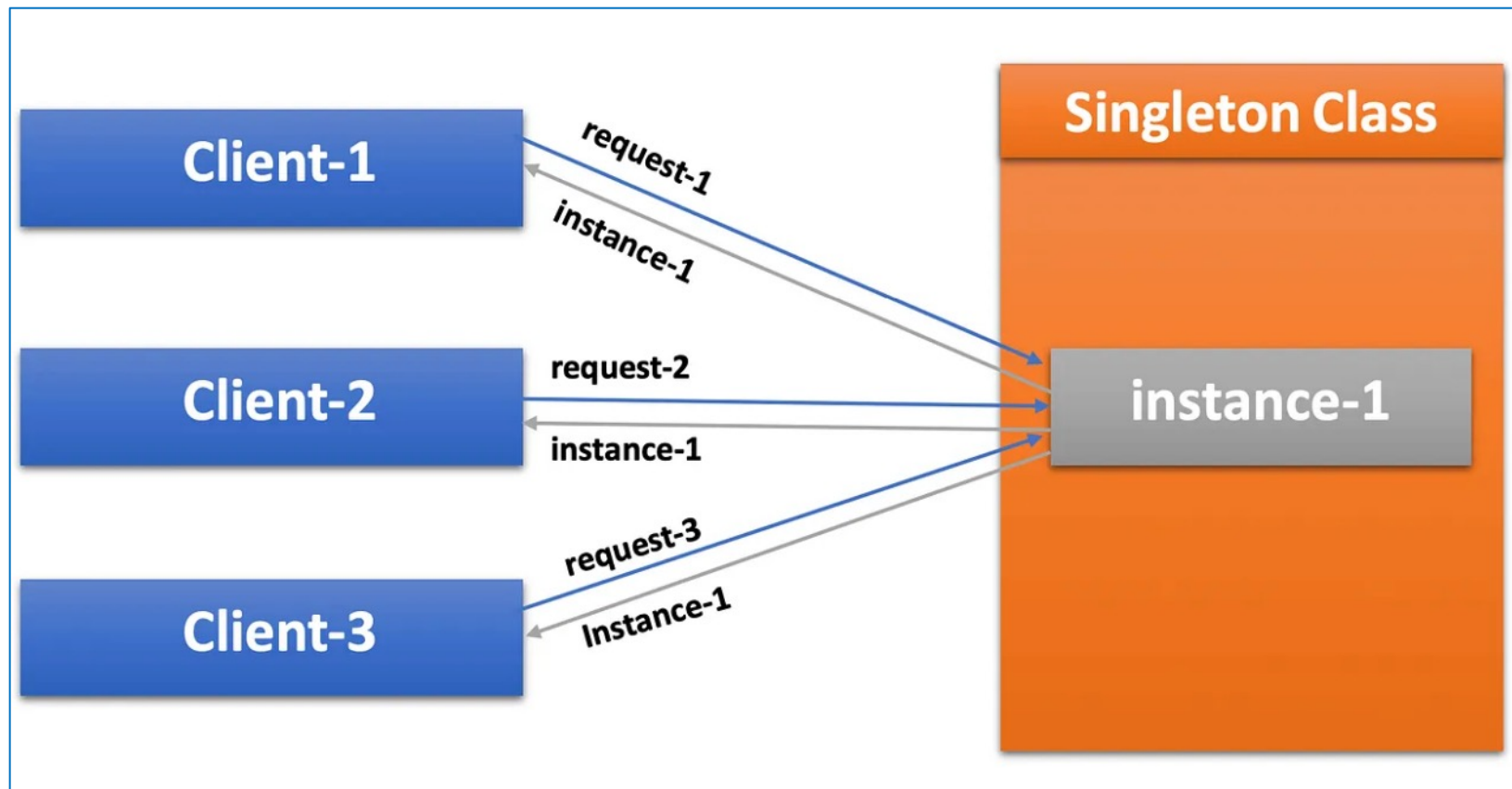
Creational Patterns: Definition

- Creational patterns abstract the instantiation process.
- They help to make a system independent of how its objects are created, composed and represented
 - Creational patterns for classes use inheritance to vary the class that is instantiated.
 - Creational patterns for objects delegate instantiation to another objects
- Ex:
 - Singleton
 - Factory

Singleton

Creational Patterns

- Ensure that only one instance of a class is created and provide a global access point to the object



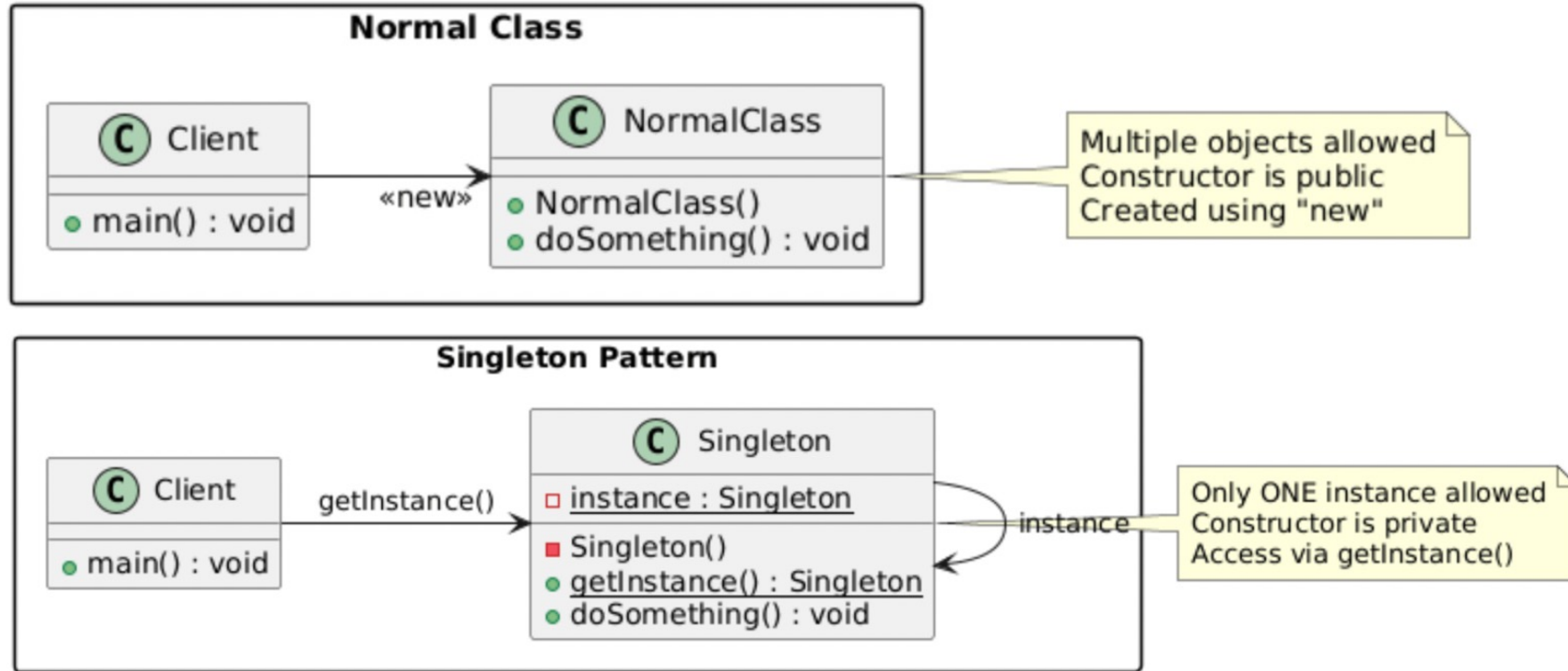
Singleton: Usage

Creational Patterns

- When to Use:
 - We must ensure that only one instance of a class is created
 - The instance must be available through all the code
 - In multi-threading environments when multiple threads must access the same resources through the same singleton object.
- Common Usage:
 - Logger Classes
 - Configurations Classes
 - Accessing resources in shared mode
 - Other design patterns implemented as Singleton: Factories and Abstract Factories, Builder, Prototype

Singleton – How to implement?

Creational Patterns



- **private static** member: instance
- **private** constructor: Singleton()
- **public** factory method: getInstance()

Singleton – Eager initialization

Creational Patterns

```
package singleton;

public class Singleton {
    // Static member to hold the single instance
    private static Singleton instance = new Singleton();

    // Private constructor to prevent external instantiation
    private Singleton() { }

    // Static factory method for global access
    public static Singleton getInstance() {
        return instance;
    }
}
```

Singleton – Lazy initialization

Creational Patterns

```
package singleton;

public class Singleton {

    private static Singleton instance;

    private Singleton() { }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Singleton – Thread Safe

Creational Patterns

```
package singleton;

public class Singleton {

    private static volatile Singleton instance;

    private Singleton() { }

    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }

        return instance;
    }
}
```


Singleton – Thread Safe – Check Locking

Creational Patterns

```
public class Singleton {
    private static volatile Singleton instance;
    private Singleton() { }
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                // Re-check again. Maybe another thread has
initialized before
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

Singleton – Bill Pugh

Creational Patterns

```
package singleton;

public class BillPughSingleton {
    private static class SingletonHelper {
        static final BillPughSingleton INSTANCE = new
BillPughSingleton();
    }

    private BillPughSingleton() { }

    public static BillPughSingleton getInstance() {
        return SingletonHelper.INSTANCE;
    }
}
```

Singleton – Serializale

Creational Patterns

```
public class SingletonSerializable implements Serializable {
    private static class SingletonHelper {
        static final SingletonSerializable INSTANCE = new
SingletonSerializable();
    }

    private SingletonSerializable() { }
    public static SingletonSerializable getInstance() {
        return SingletonHelper.INSTANCE;
    }

    private Object readResolve() {
        return SingletonHelper.INSTANCE;
    }
}
```

Singleton – Enum

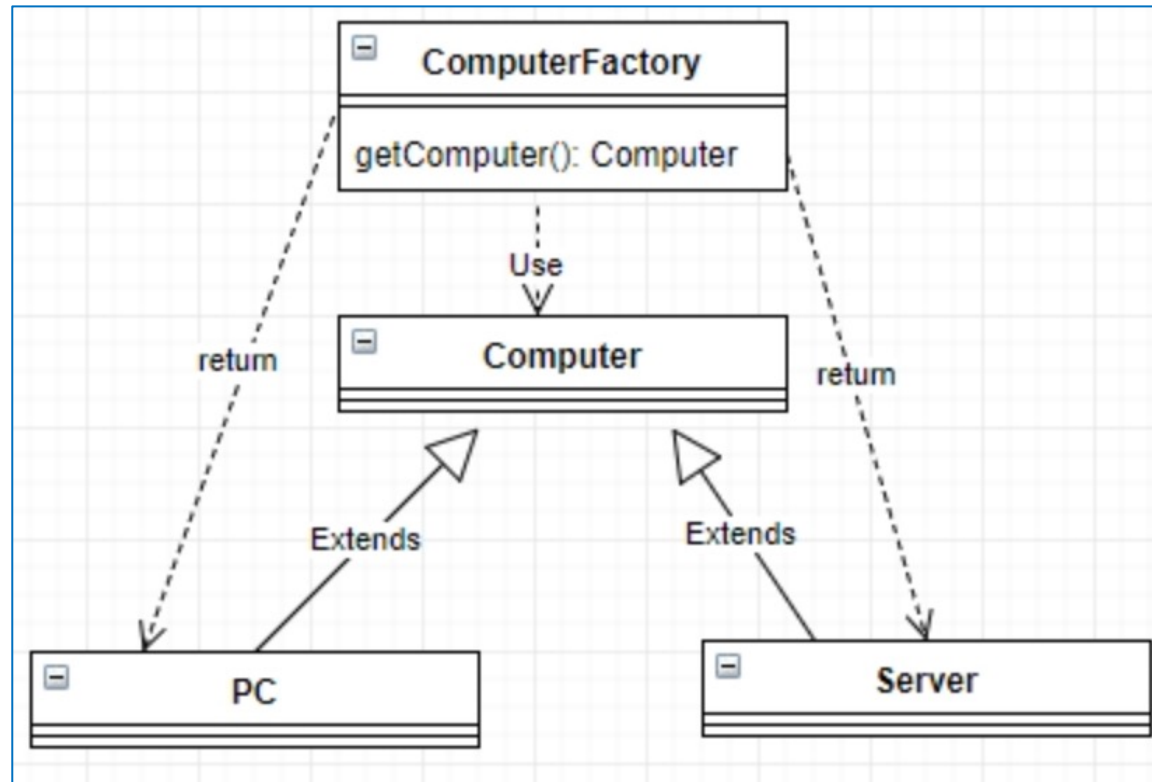
Creational Patterns

```
public enum SingletonEnum {  
  
    INSTANCE;  
  
    int value;  
  
    public int getValue() {  
        return value;  
    }  
  
    public void setValue(int value) {  
        this.value = value;  
    }  
}
```

Factory Method

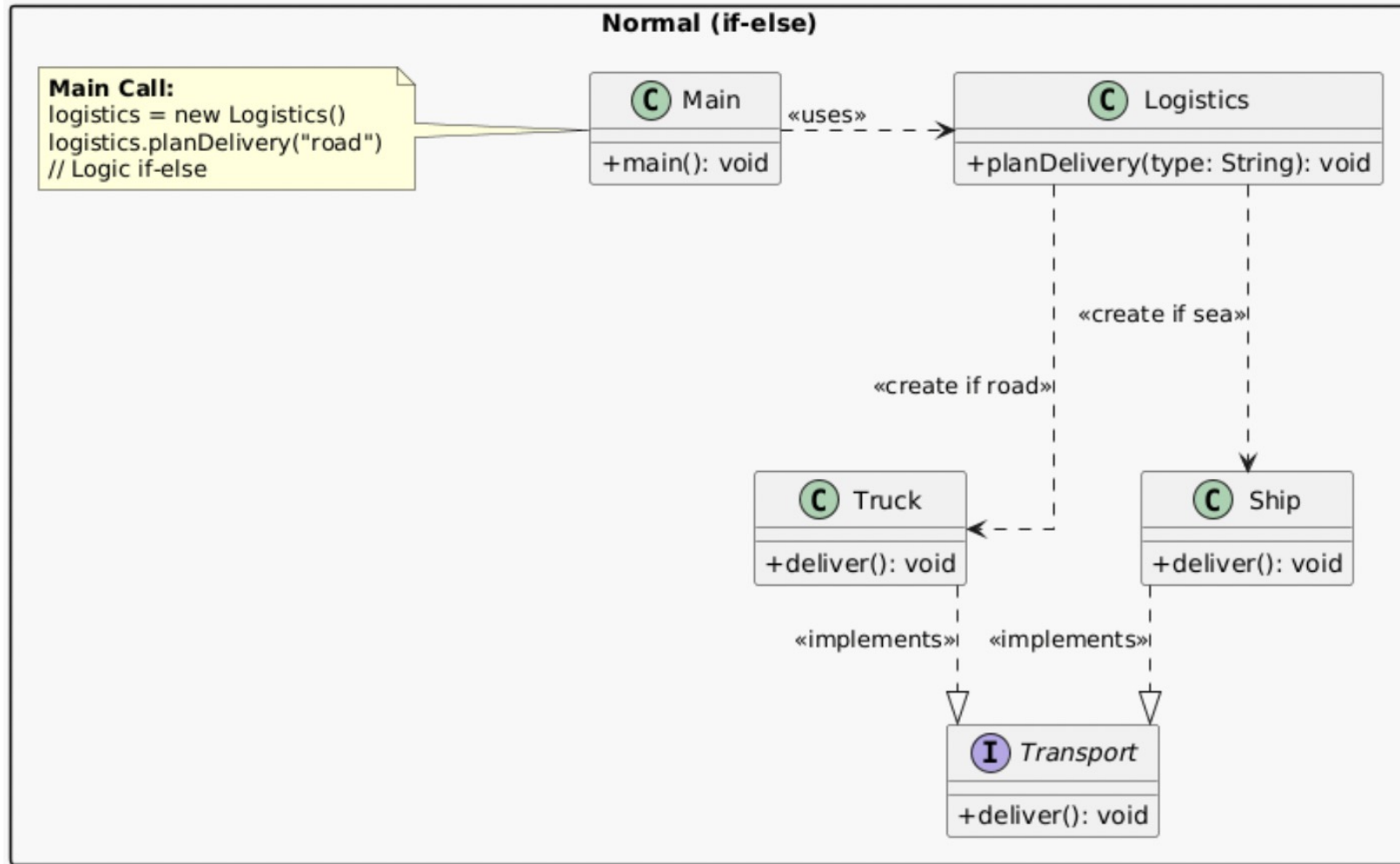
Creational Patterns

- Provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



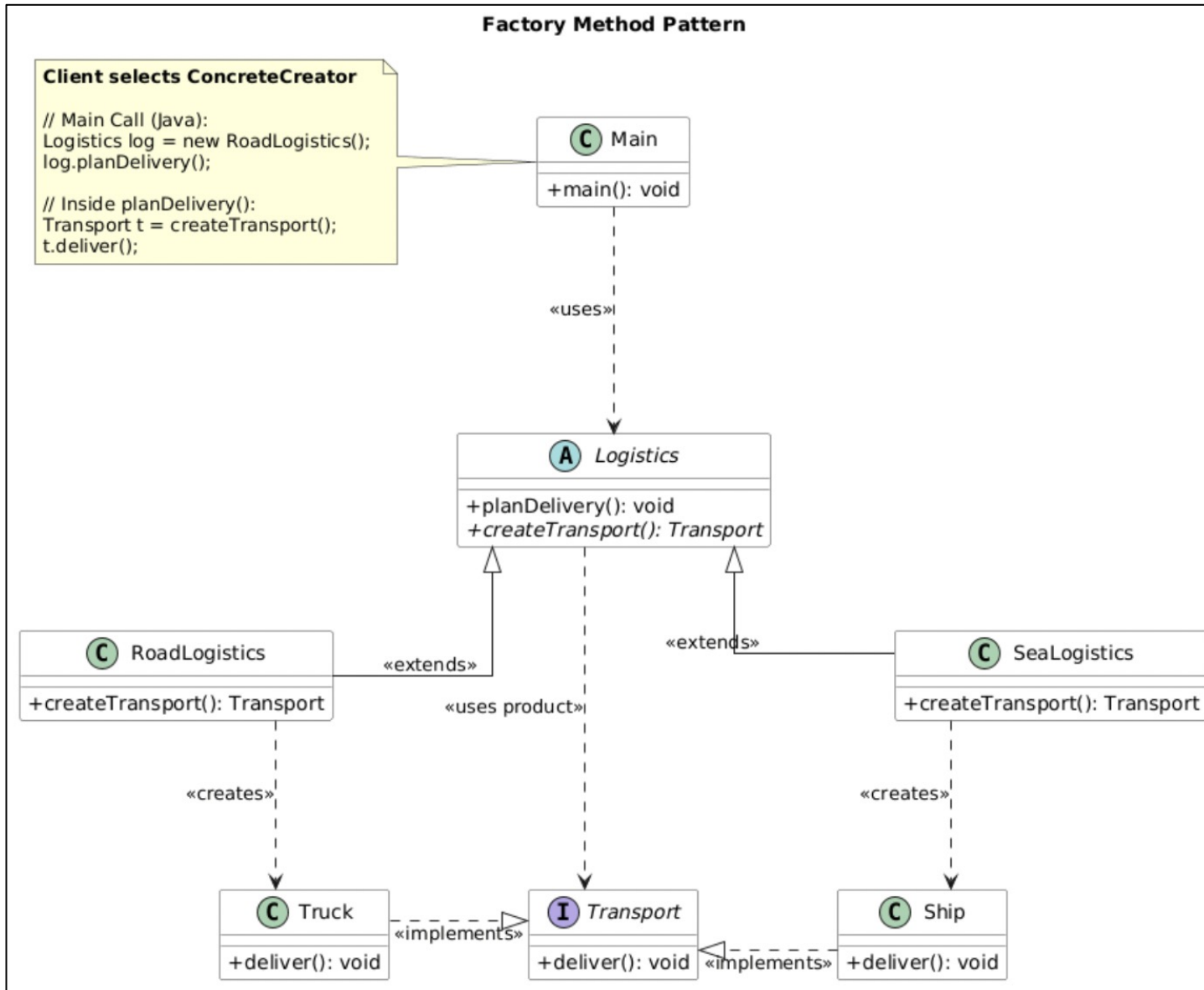
Factory Method (cont.)

Creational Patterns



Factory Method (cont.)

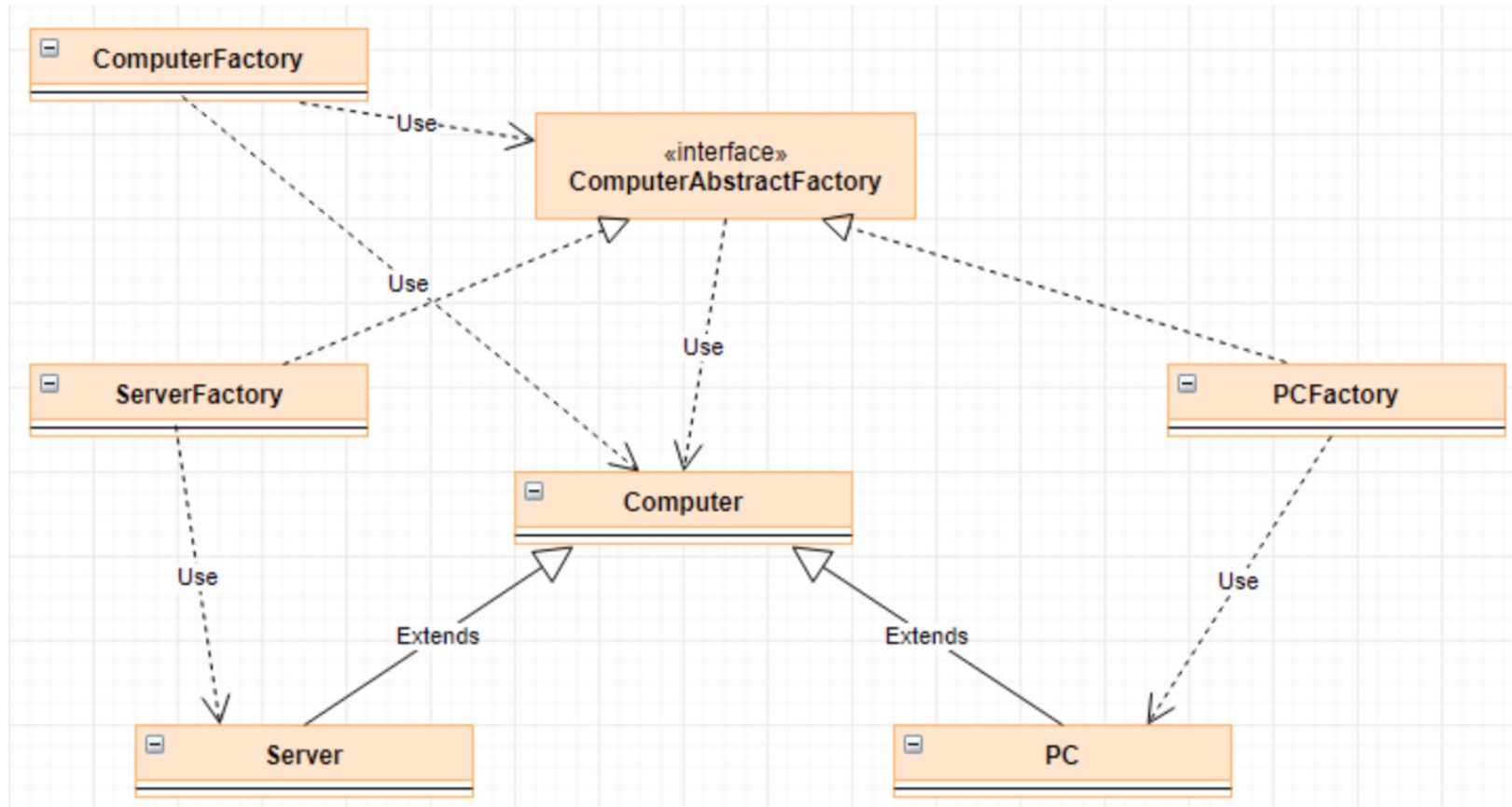
Creational Patterns



- **Product (Interface) - Transport:** định nghĩa hành vi chung
 - **Concrete Products - Truck, Ship:** các lớp thực thi cụ thể
 - **Creator (Abstract Class) - Logistics :** khai báo factory method `createTransport()` và sử dụng Product trong nghiệp vụ → Creator định nghĩa factory method nhưng không quyết định sản phẩm cụ thể.
 - **Concrete Creators:** Override factory method để tạo ra ConcreteProduct tương ứng.
- **Factory Method Pattern** định nghĩa một interface để tạo đối tượng, nhưng cho phép các subclass quyết định lớp cụ thể sẽ được khởi tạo.
- Creator (Logistics) dùng Product (Transport) nhưng không biết ConcreteProduct (Truck, Ship) đó là ý nghĩa chính của Factory Method.

Abstract Factory

Creational Patterns

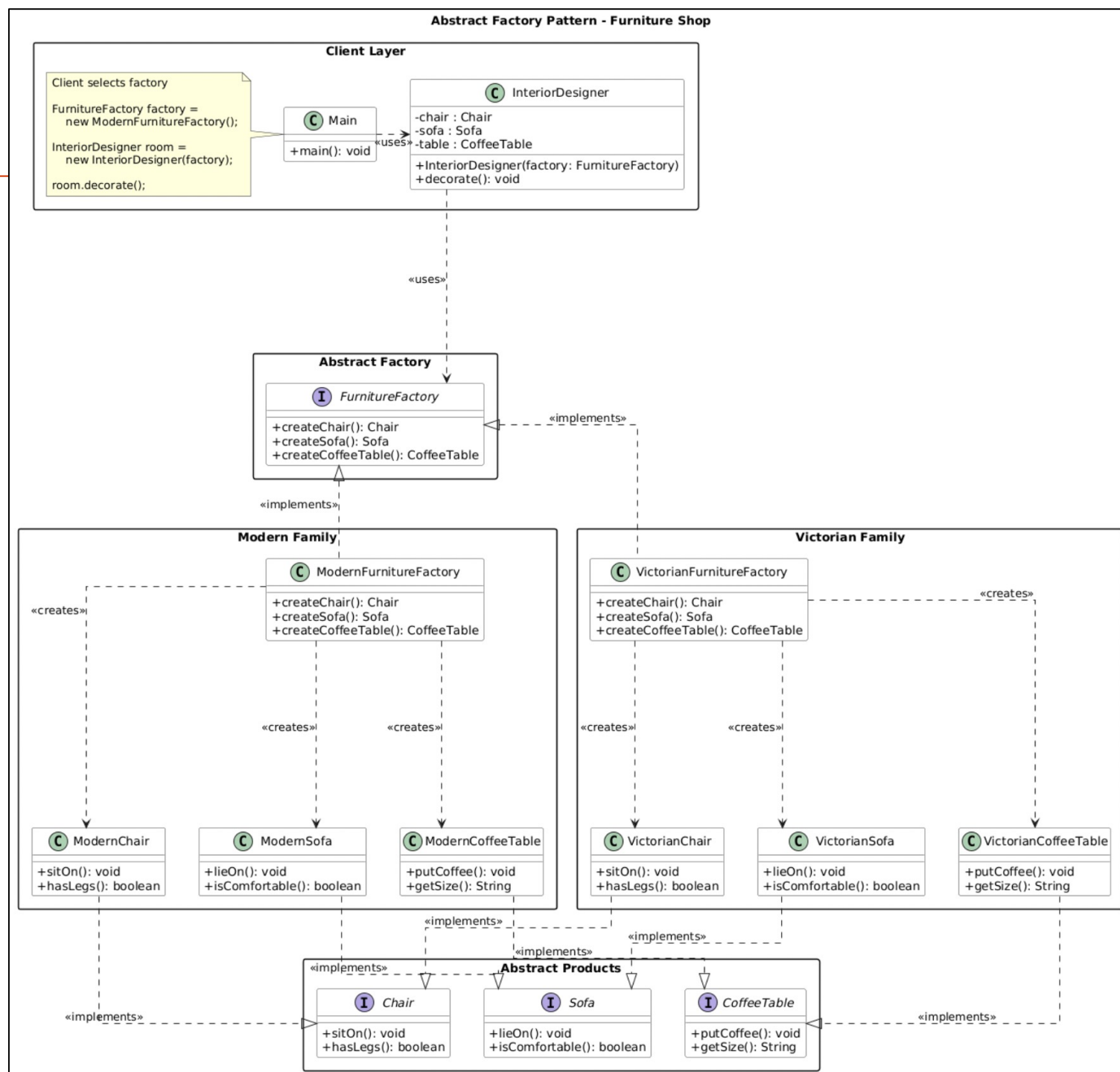


Abstract Factory

Creational Patterns

- **Abstract Product (Interface):** định nghĩa cho từng loại Product. trong 1 Product family (Chair, Sofa, CoffeeTable)
- **Concrete Products:** các biến thể cụ thể (ModernChair, VictorianChair, ...)
- **Abstract Factory (Interface):** khai báo các phương thức để tạo ra từng Product. Các product được tạo ra luôn cùng một family.
- **Concrete Factories:** các lớp thực thi factory, mỗi factory tạo ra một **product family** cụ thể (ModernFurnitureFactory, VictorianFurnitureFactory)

→ **Abstract Factory Pattern** cung cấp một interface để tạo ra các họ đối tượng liên quan mà không cần chỉ định lớp cụ thể của chúng.



Structural Patterns: Definition

- Structural patterns are concerned with how classes and objects are composed to form larger structures
 - Structural class patterns use inheritance to compose interfaces or implementations
 - Structural object patterns describe ways to compose objects to realize new functionality. The added flexibility of object composition comes from the ability to change the composition at runtime, which is impossible with static class composition
- Ex:
 - Adapter
 - Composite
 - Decorator

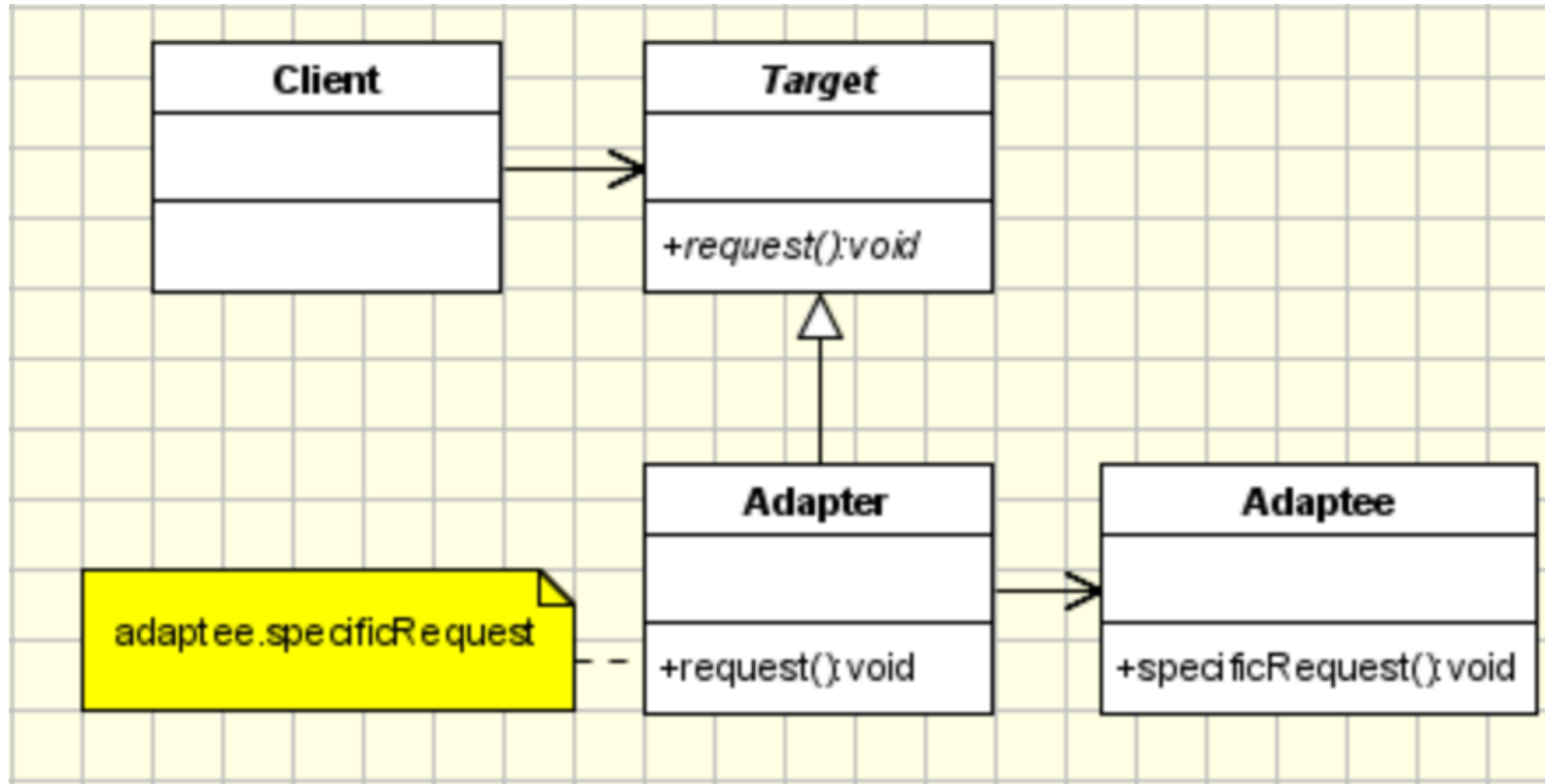
Adapter

Structural Patterns

- Converts the interface of a class into another interface the clients expect
- Adapter lets classes work together, that could not otherwise because of incompatible interfaces.

Adapter - Implementation

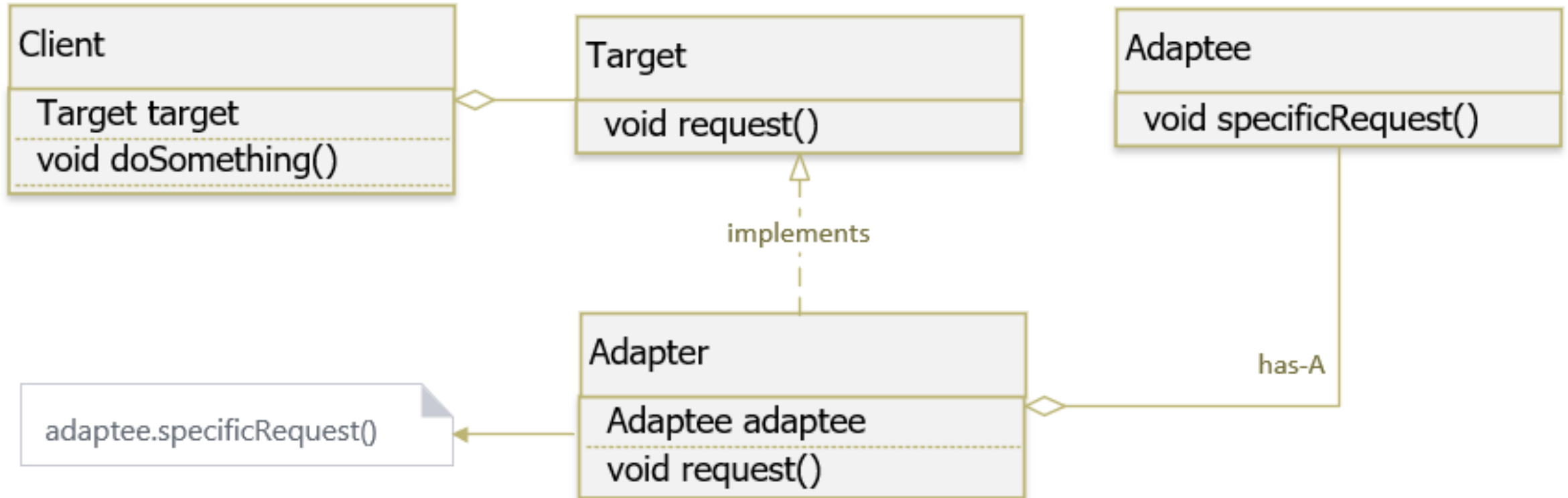
Structural Patterns



- **Target:** defines the domain-specific interface that Client uses.
- **Adapter:** adapts the interface Adaptee to the Target interface.
- **Adaptee:** defines an existing interface that needs adapting.
- **Client:** collaborates with objects conforming to the Target interface.

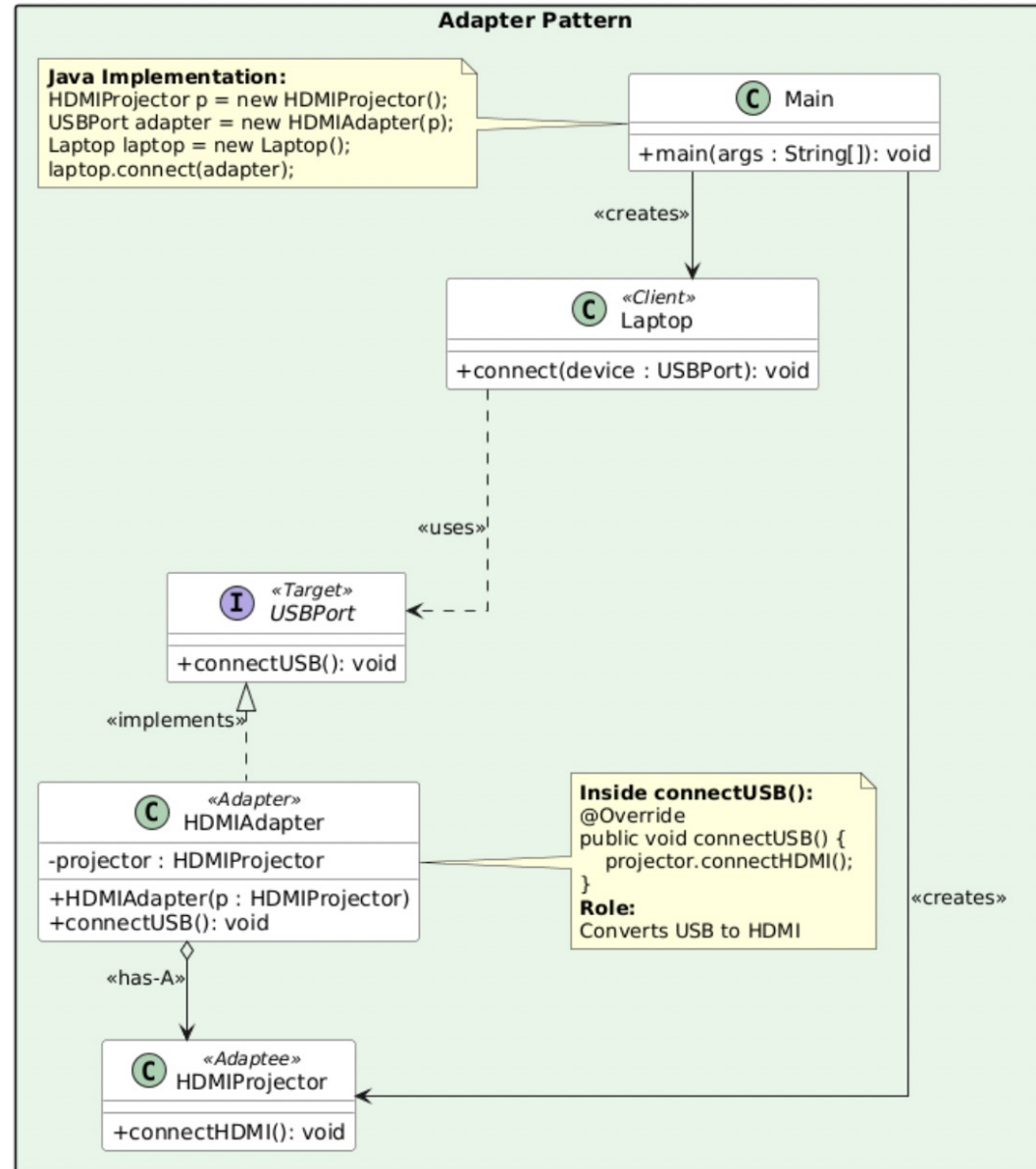
Adapter – Object Adapter – Composition (Chứa trong)

Structural Patterns



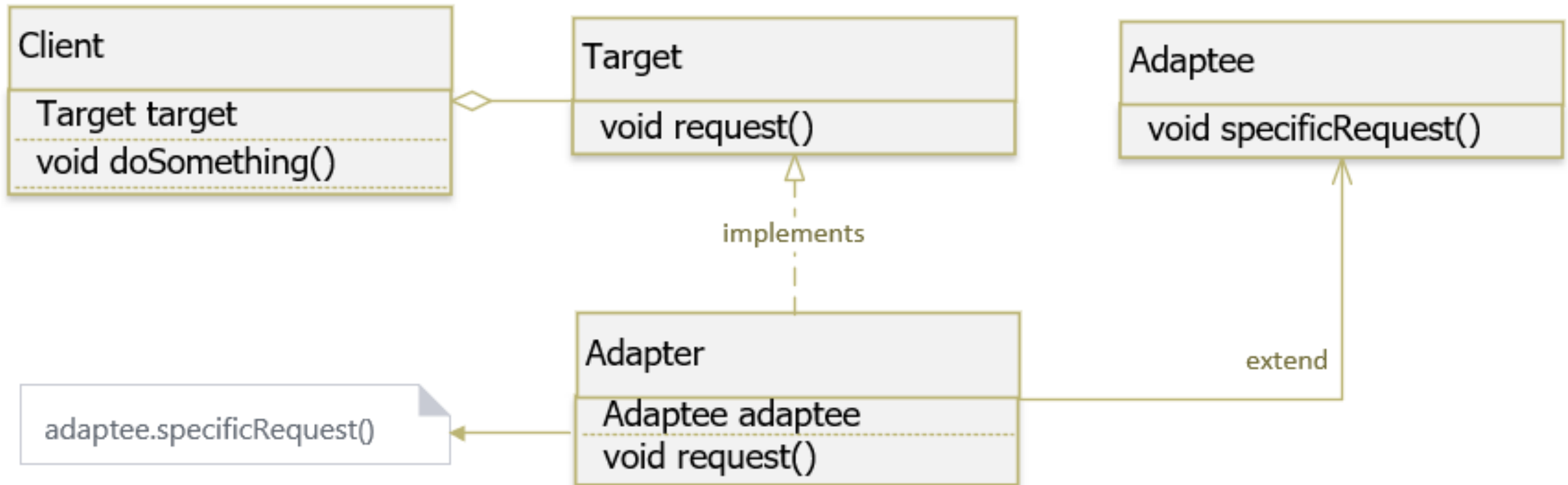
Adapter – Object Adapter – Composition (cont.)

Structural Patterns



Adapter – Class Adapter – Inheritance

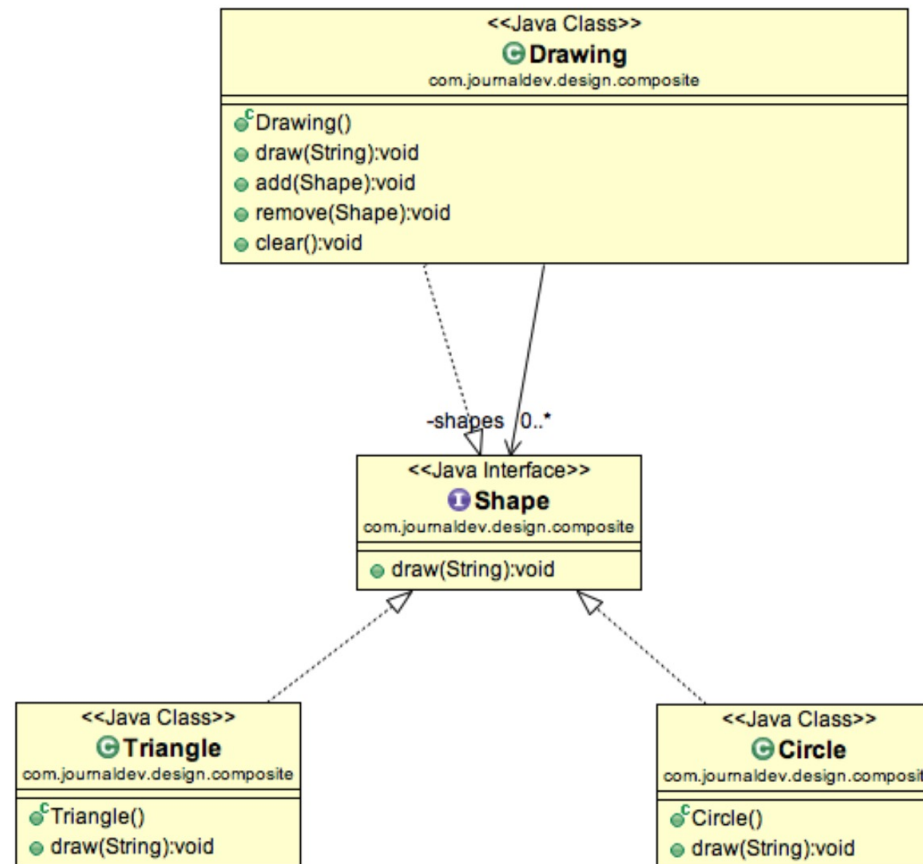
Structural Patterns



Composite

Structural Patterns

Takes a group of objects into a single object. This pattern is perfect for representing part-whole hierarchies, such as a graphics system where shapes like circles and squares can be grouped into a larger diagram.



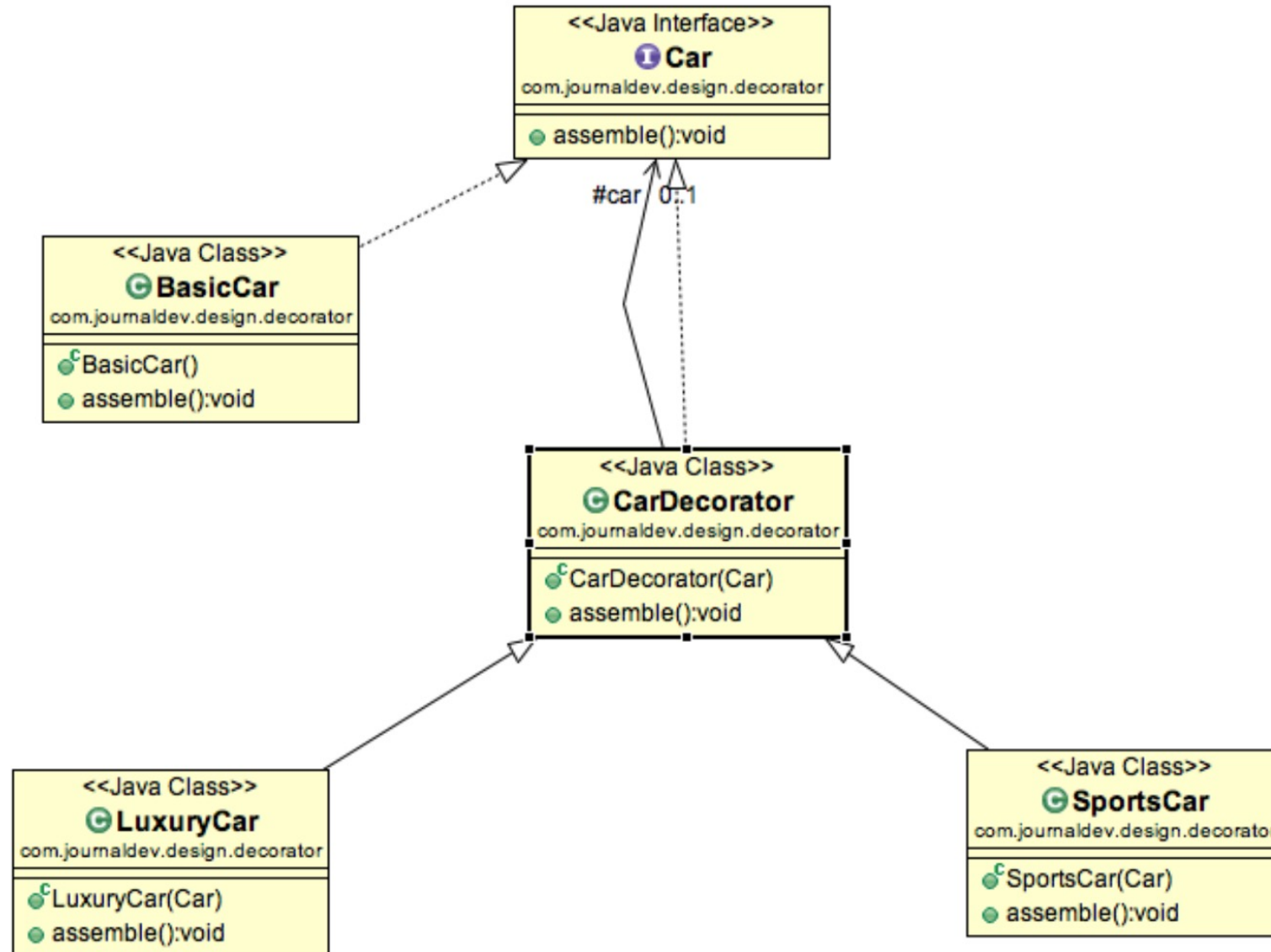
Decorator

Structural Patterns

Decorator design pattern is used to modify the functionality of an object at runtime. At the same time other instances of the same class will not be affected by this, so individual object gets the modified behavior. Decorator design pattern is one of the structural design pattern (such as [Adapter Pattern](#), [Bridge Pattern](#), [Composite Pattern](#)) and uses abstract classes or interface with [composition](#) to implement.

Decorator

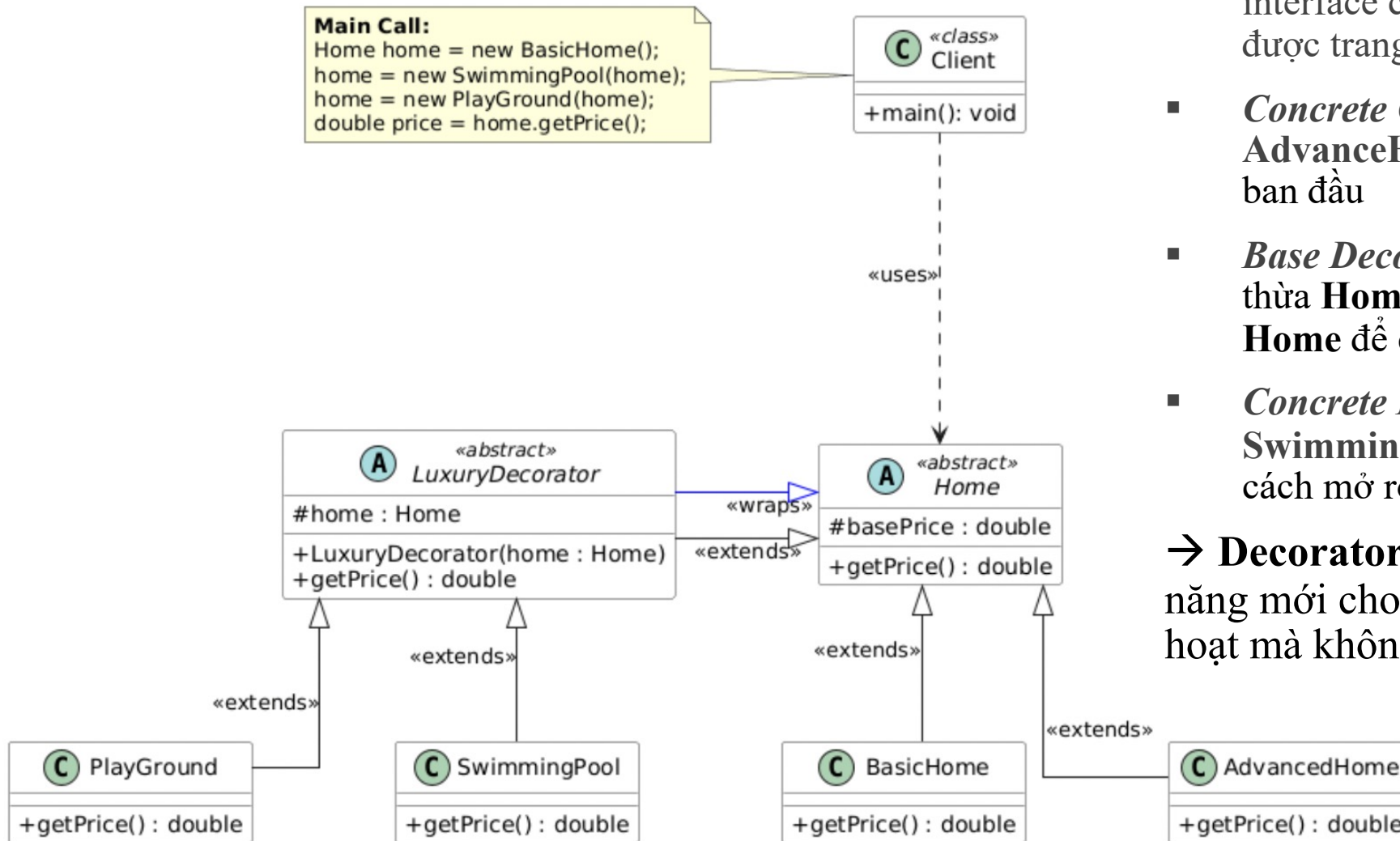
Structural Patterns



Decorator (cont.)

Structural Patterns

Decorator Pattern - Home Pricing System



- **Component (abstract) - Home:** định nghĩa interface chung cho các đối tượng có thể được trang trí.
- **Concrete Components - BasicHome, AdvanceHome:** cung cấp chức năng cơ bản ban đầu
- **Base Decorator - LuxuryDecorator:** kế thừa **Home** và chứa một tham chiếu tới **Home** để có thể bọc (wraps) đối tượng khác.
- **Concrete Decorators - PlayGround, SwimmingPool:** thêm hành vi mới bằng cách mở rộng **LuxuryDecorator**

→ **Decorator Pattern** cho phép thêm chức năng mới cho một đối tượng một cách linh hoạt mà không cần sửa đổi class gốc.

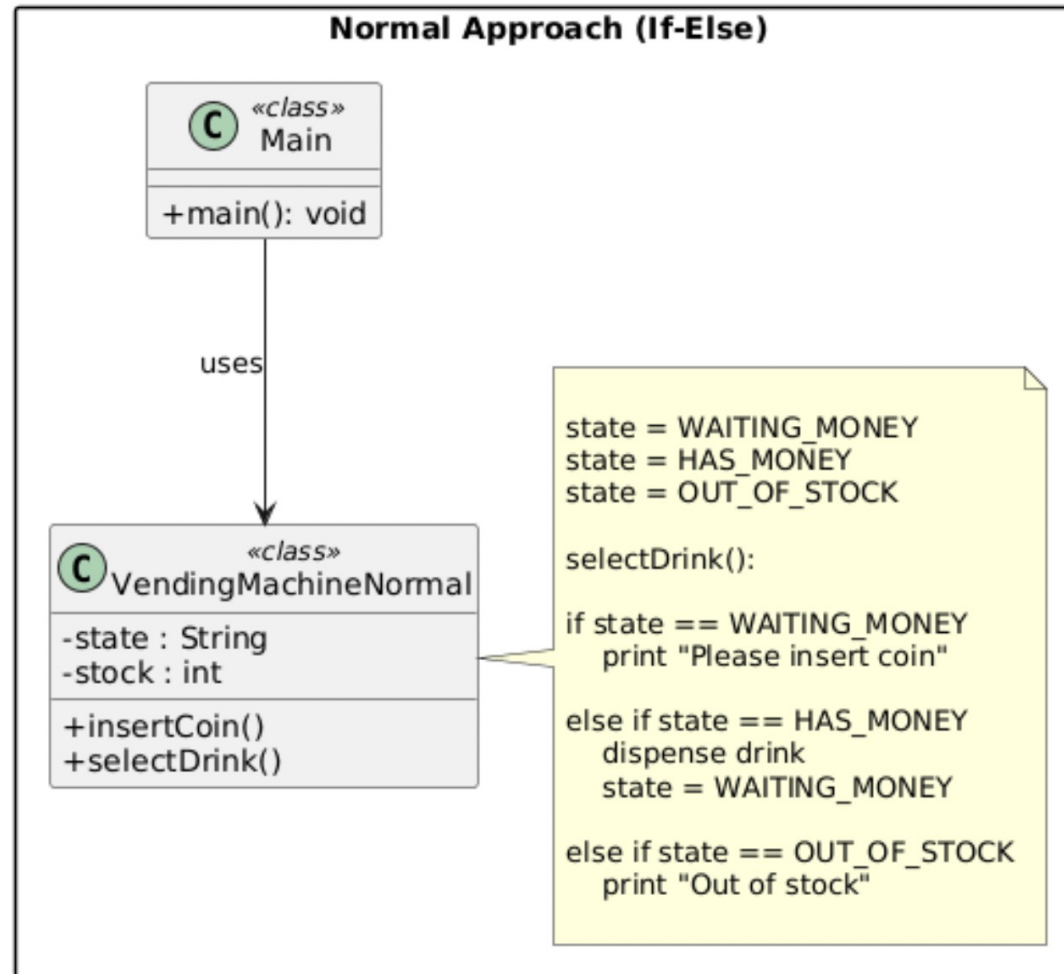
Behavioral Patterns: Definition

- Structural patterns are concerned with algorithms and the assignment of responsibilities between objects
 - Behavioral class patterns use inheritance to distribute behavior between classes
 - Behavioral object patterns use composition rather than inheritance. Some describe how a group of peer objects cooperate to perform a task that no single object can carry out by itself.
- Ex:
 - State
 - Strategy
 - Chain Of Responsibility
 - Observer

State

Behavioral Patterns

State pattern is one of the **behavioral design pattern**. State design pattern is used when an Object change its behavior based on its internal state.

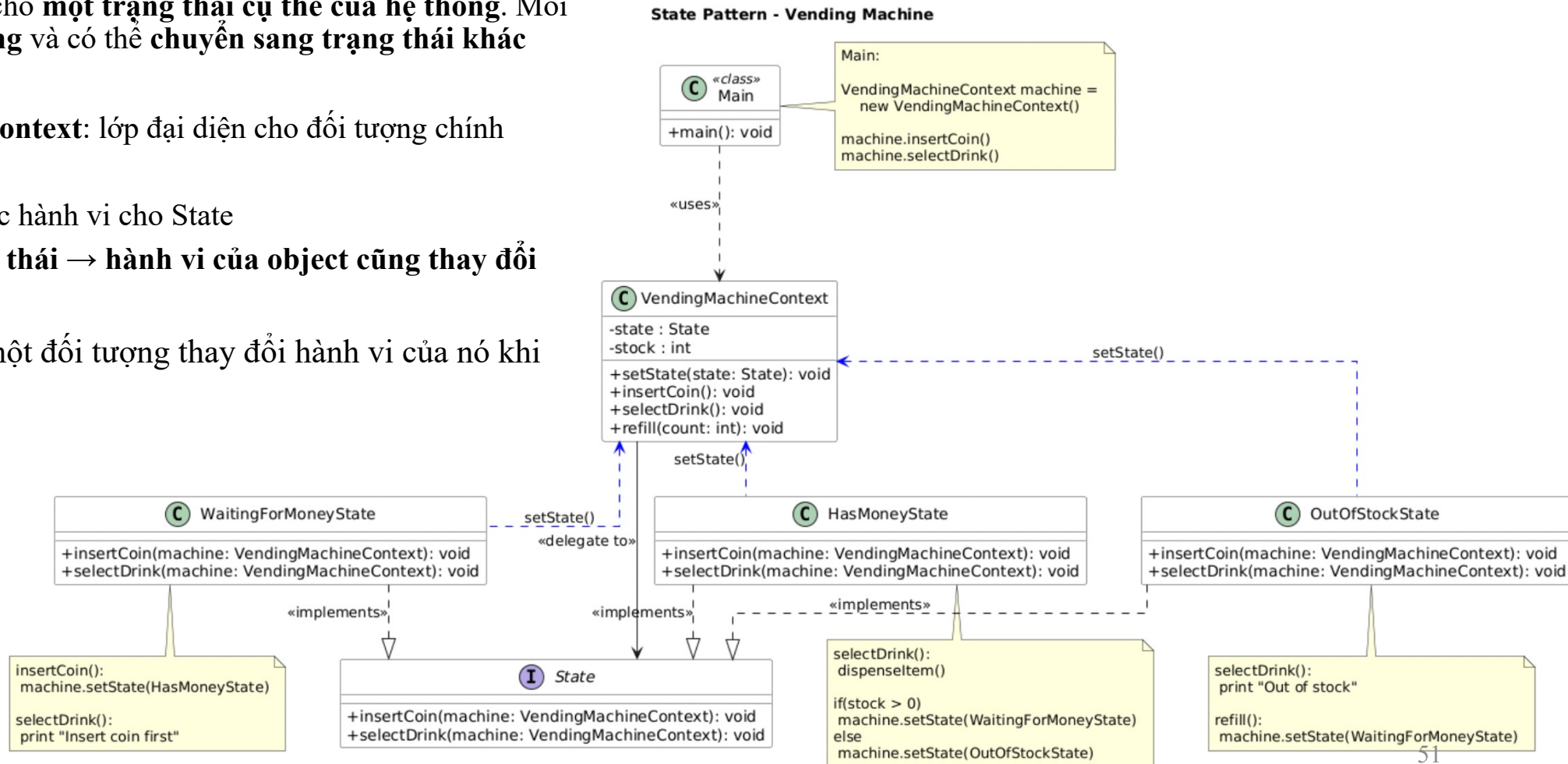


State (cont.)

Behavioral Patterns

- **State (Interface) - State:** định nghĩa hành vi chung.
- **Concrete State – WaitingForMoneyState, HasMoneyState, OutOfStockState:** đại diện cho **một trạng thái cụ thể của hệ thống**. Mỗi class định nghĩa **hành vi riêng** và có thể **chuyển sang trạng thái khác** bằng setState()
- **Context - VendorMachineContext:** lớp đại diện cho đối tượng chính
 - giữ **state hiện tại**
 - **ủy quyền (delegate)** các hành vi cho State
 - cho phép **chuyển trạng thái** → hành vi của object cũng thay đổi

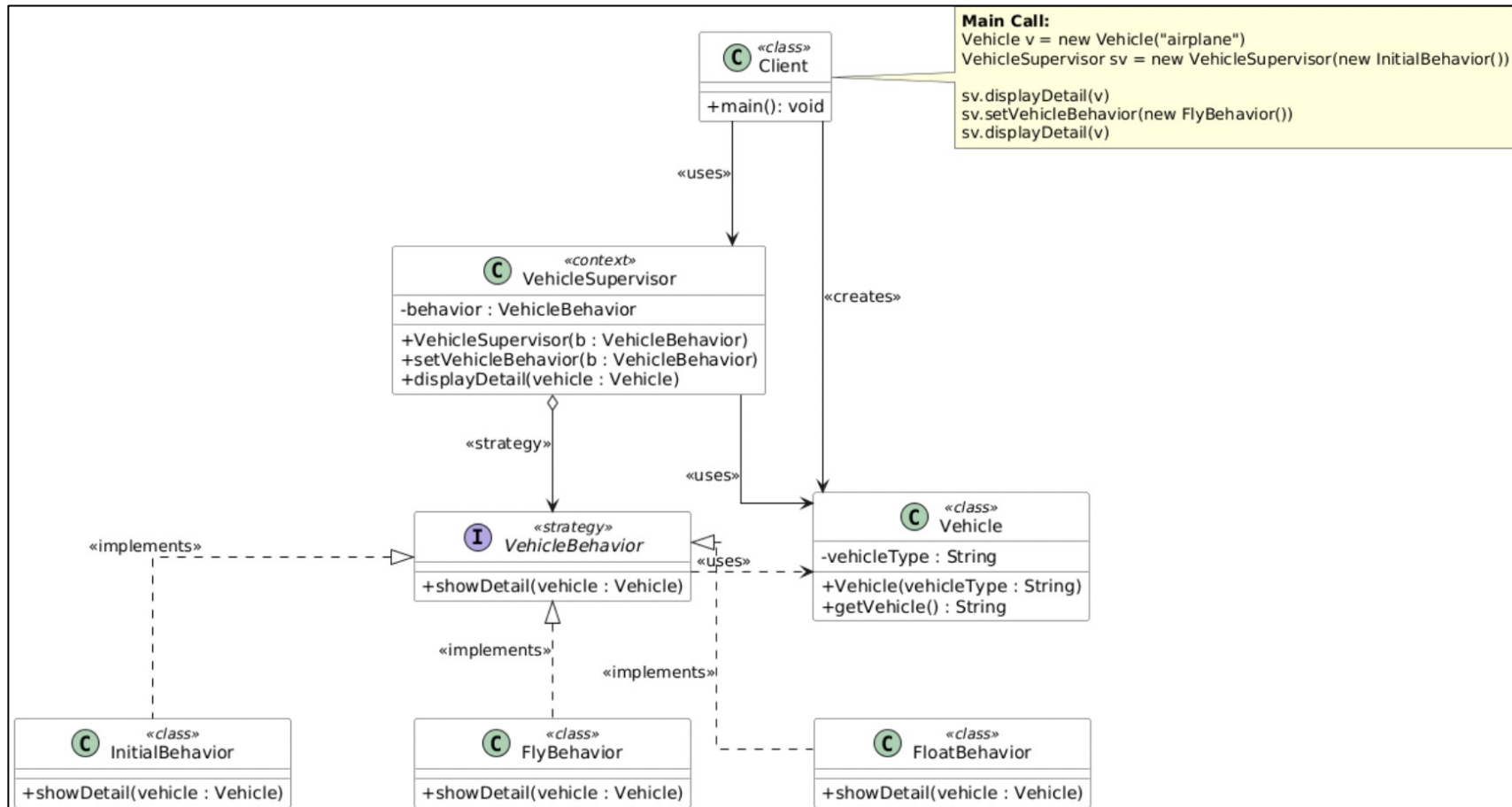
→ **State Pattern** cho phép một đối tượng thay đổi hành vi của nó khi trạng thái nội tại thay đổi.



Strategy

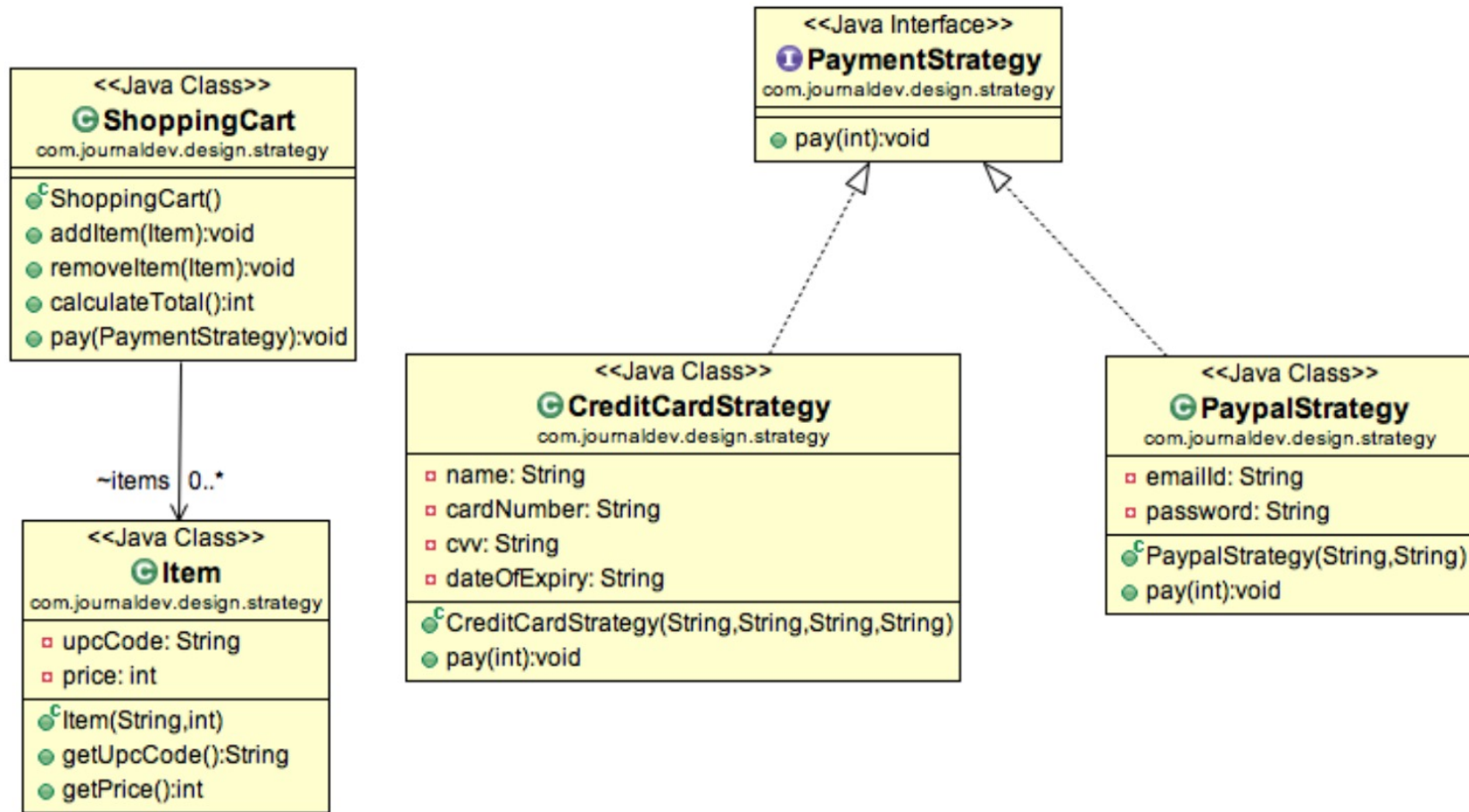
Behavioral Patterns

Strategy pattern is one of the **behavioral design pattern**. Strategy pattern is used when we have multiple algorithm for a specific task and client decides the actual implementation to be used at runtime.



Strategy (cont.)

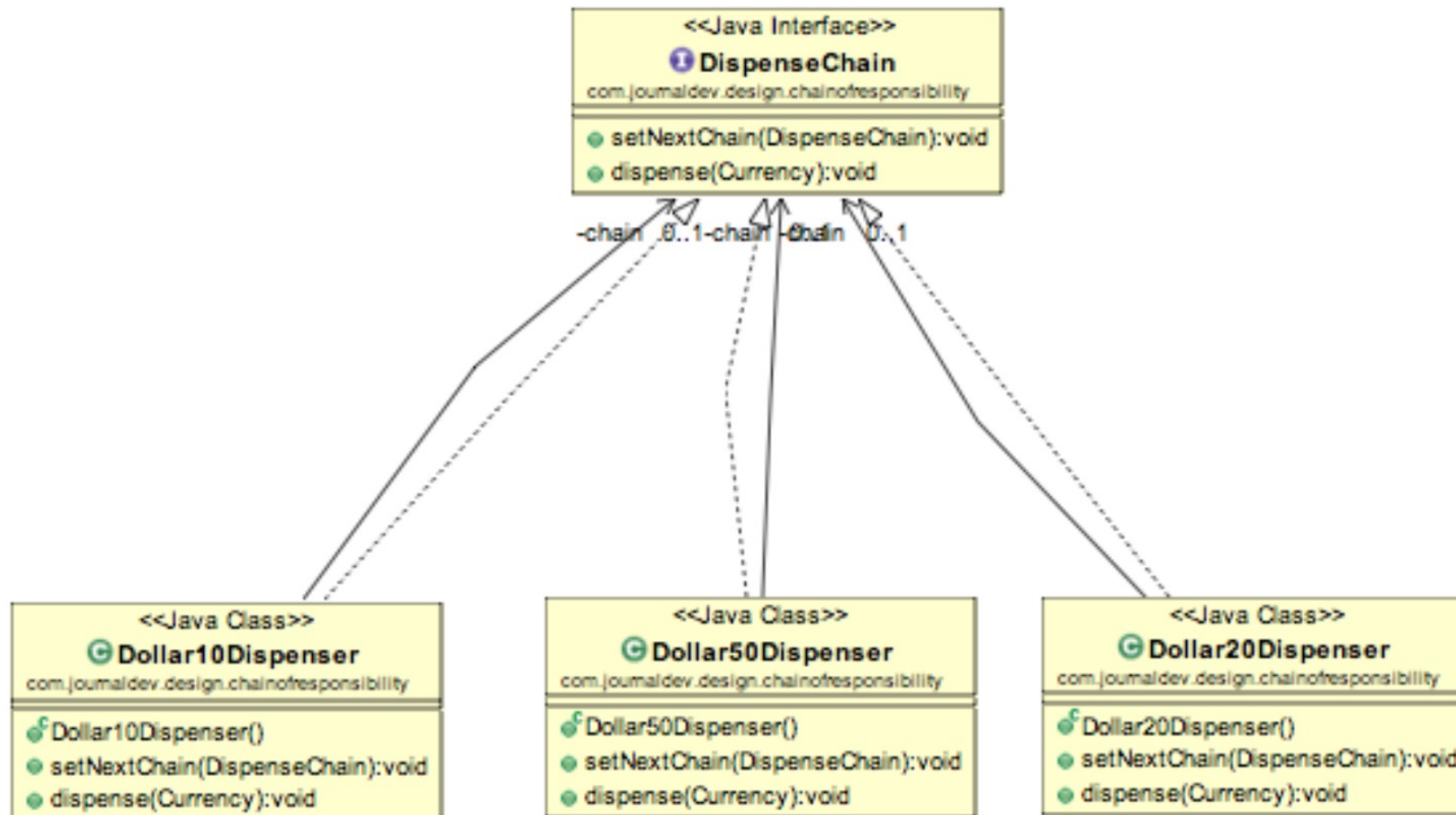
Behavioral Patterns



Chain of Responsibility

Behavioral Patterns

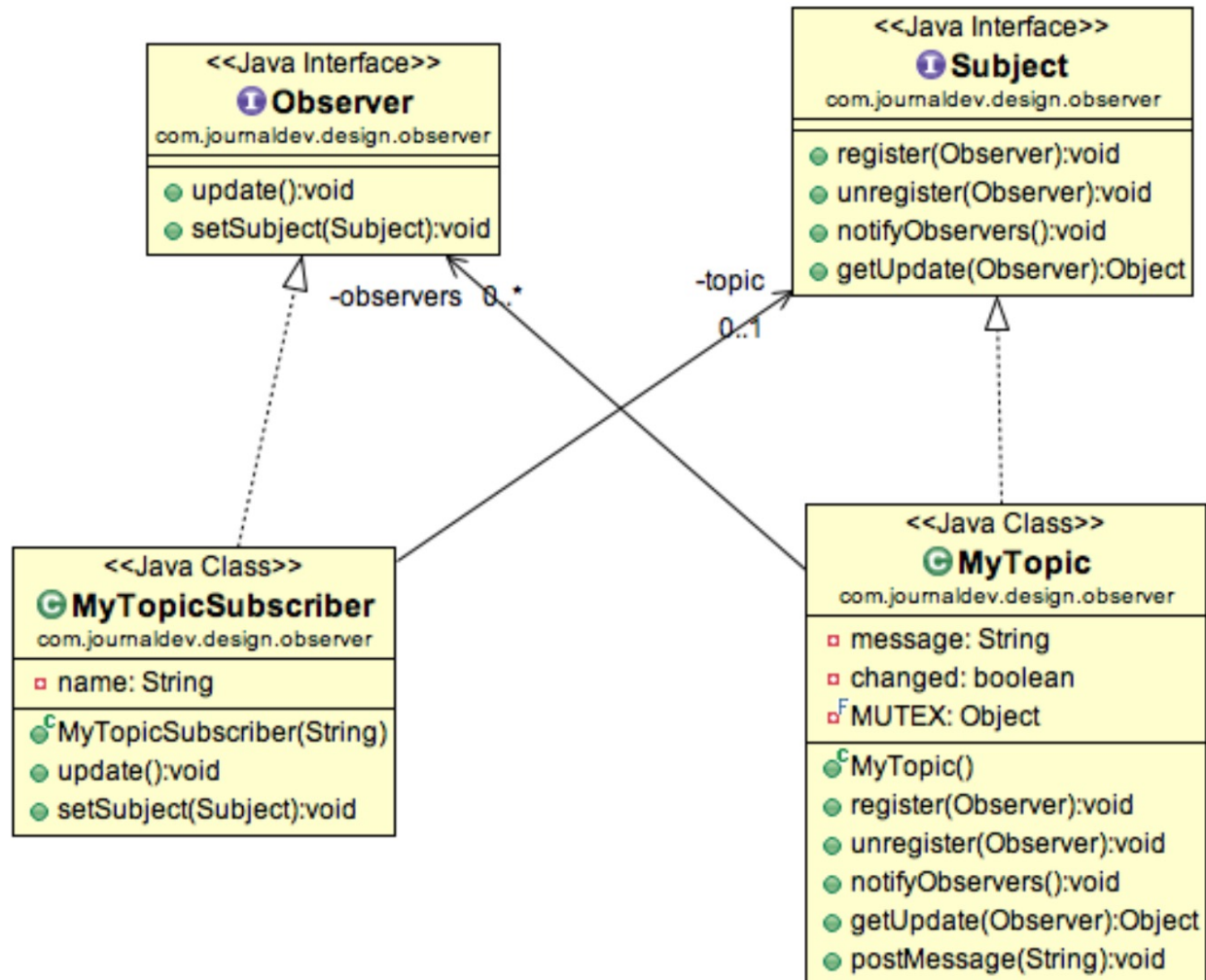
The **Chain of Responsibility design pattern** is a behavioral design pattern that allows an object to pass a request along a chain of handlers.



Observer

Behavioral Patterns

Define a one-to-many dependency between objects where a state change in one object results with all its dependents being notified and updated automatically



Q&A